

TECNOLOGICO JOSE MARIO MOLINA PASQUEL Y HENRÍQUEZ UA ZAPOTLANEJO

MANUAL DE PROGRAMADOR

DESARROLLO DE APLICACIONES PARA DISPOSITIVOS MÓVILES

JOSE LUIS OLIVARES ACEVES

INFO 5



ÍNDICE

CONTENIDO

PRÁCTICA 1 ..... **3**
PRÁCTICA 2 ..... **5**
PRÁCTICA 3 ..... **7**
PRÁCTICA 4 ..... **9**
PRÁCTICA 5 ..... **11**
PRÁCTICA 6 ..... **13**
UNIDAD 2 ..... **16**
PRÁCTICA 7 ..... **16**
PRÁCTICA 8 ..... **18**
PRÁCTICA 9 ..... **20**
PRÁCTICA 10 ..... **22**



PRÁCTICA 1

main.dart:

practica1 > lib > main.dart > MyApp

```dart
import 'package:flutter/material.dart'; // Importa componentes de diseño Material de Flutter / Imports Flutter Material design components
import 'package:practica1/operaciones/practica4.dart'; // Importa el módulo de la práctica 4 / Imports the practice 4 module
import 'package:practica1/operaciones/suma.dart'; // Importa la función u operaciones de suma / Imports the addition operations or function
import 'package:practica1/widget/inicio.dart'; // Importa el widget de la pantalla de inicio / Imports the home screen widget
import 'package:practica1/widget2/practica2.dart'; // Importa el widget de la práctica 2 / Imports the practice 2 widget

Run | Debug | Profile
void main() { // Función principal que inicia la ejecución / Main function that starts execution
  runApp(const MyApp()); // Infla los widgets y lanza la aplicación / Inflates widgets and launches the application
}

class MyApp extends StatelessWidget { // Define la raíz de la aplicación como widget sin estado / Defines the app root as a stateless widget
  const MyApp({super.key}); // Constructor de la clase con clave de identificación / Class constructor with identification key

  @override // Sobrescribe el método de construcción de la clase padre / Overrides the parent class build method
  Widget build(BuildContext context) { // Método para construir el árbol de interfaz gráfica / Method to build the UI widget tree
    return MaterialApp( // Crea la estructura global basada en Material Design / Creates the global structure based on Material Design
      home: Inicio(), // Define el widget 'Inicio' como pantalla principal / Sets the 'Inicio' widget as the main screen
    ); // MaterialApp
  }
}
```

inicio.dart:

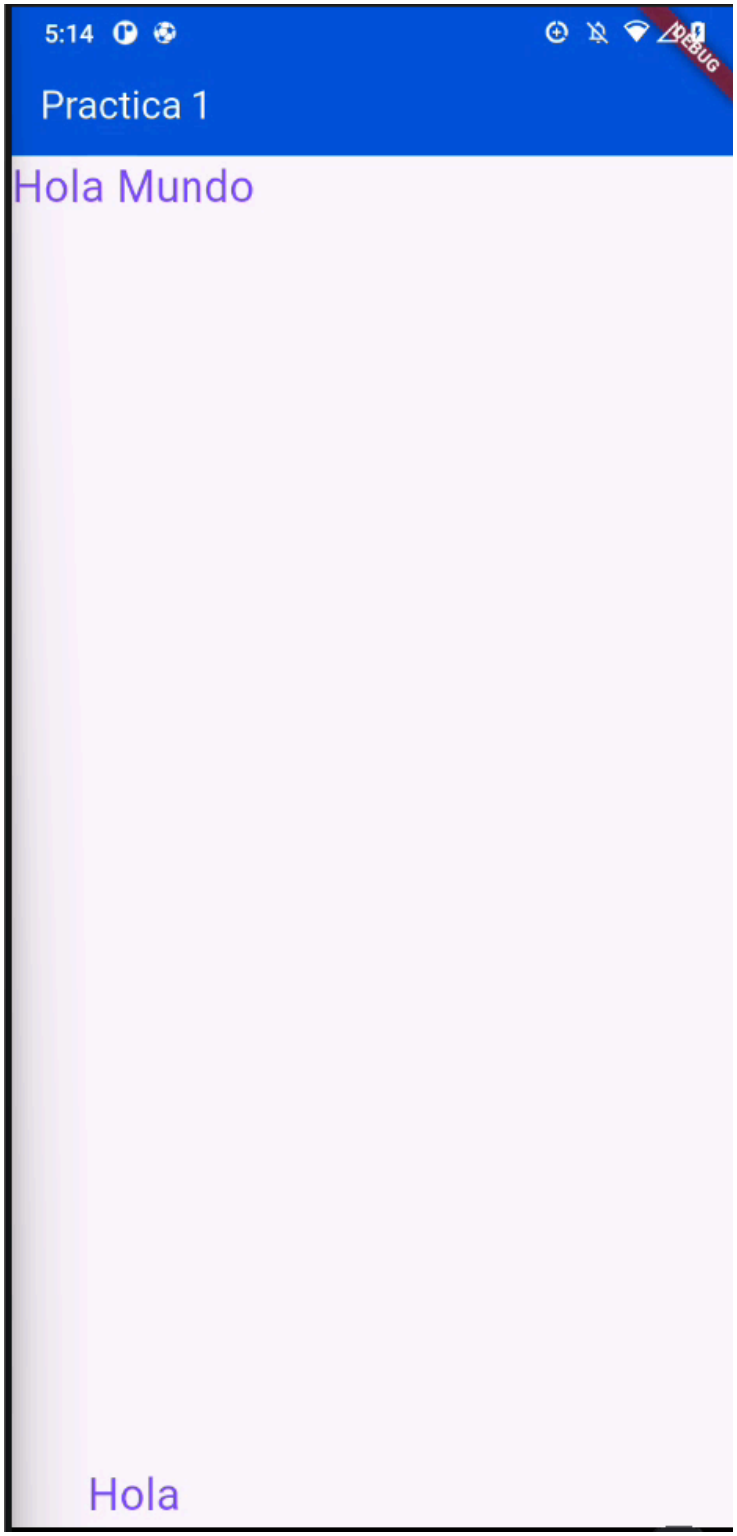
practica1 > lib > widget > inicio.dart > Disenio

```dart
import 'package:flutter/material.dart'; // Importa los componentes de interfaz gráfica de Flutter / Imports Flutter UI components

class Inicio extends StatefulWidget { // Define un widget con estado dinámico / Defines a widget with dynamic state
  @override // Sobrescribe el método de creación del estado / Overrides the state creation method
  State<StatefulWidget> createState() { // Crea el estado asociado al widget Inicio / Creates state associated with Inicio widget
    return Disenio(); // Retorna la clase que maneja la interfaz y el estado / Returns the class managing UI and state
  }
}

class Disenio extends State<Inicio> { // Clase de estado que controla el widget Inicio / State class controlling the Inicio widget
  @override // Sobrescribe el método de construcción de la interfaz / Overrides the UI build method
  Widget build(BuildContext context) { // Método para construir la estructura visual / Method to build the visual structure
    return Scaffold( // Proporciona la estructura básica de la pantalla / Provides basic layout structure for the screen
      appBar: AppBar( // Configura la barra superior de la aplicación / Configures the top app bar
        title: Text("Practica 1", // Define el texto del título en la barra / Sets the title text in the app bar
          style: TextStyle( // Aplica estilos al texto del título / Applies styles to the title text
            color: Colors.white, // Establece el color de texto a blanco / Sets text color to white
          ), // TextStyle
        ), // Text
        backgroundColor: const Color.fromARGB(255, 6, 86, 223), // Define el color de fondo de la barra en azul / Sets app bar background color to blue
      ), // AppBar

      body: Column( // Organiza sus elementos hijos en una columna vertical / Layouts child widgets in a vertical column
        mainAxisAlignment: MainAxisAlignment.spaceBetween, // Distribuye el espacio vertical entre los elementos / Distributes space vertically between elements
        crossAxisAlignment: CrossAxisAlignment.center, // Alinea los elementos al centro horizontalmente / Centers elements horizontally
        children: [ // Lista de widgets hijos en la columna / List of child widgets in the column
          Text("Hola Mundo", // Define un texto que muestra "Hola Mundo" / Defines a text displaying "Hola Mundo"
            style: TextStyle( // Aplica estilo al primer texto / Applies style to the first text
              color: Colors.deepPurpleAccent, // Establece el color del texto en púrpura / Sets text color to deep purple
              fontSize: 25, // Define el tamaño de letra en 25 puntos / Sets font size to 25 points
            ), // TextStyle
          ), // Text
          Text("Hola", // Define un segundo texto que muestra "Hola" / Defines a second text displaying "Hola"
            style: TextStyle( // Aplica estilo al segundo texto / Applies style to the second text
              color: Colors.deepPurpleAccent, // Establece el color del texto en púrpura / Sets text color to deep purple
              fontSize: 25, // Define el tamaño de letra en 25 puntos / Sets font size to 25 points
            ), // TextStyle
          ), // Text
        ],
      ), // Column
    ); // Scaffold
  }
}
```



En esta práctica se creó la pantalla inicial de una aplicación móvil en Flutter. En la pantalla del teléfono se observa una barra superior de color azul con el título "Practica 1" en texto blanco. Debajo, en el área principal, se organizan de manera vertical dos textos centrados de color morado con el tamaño modificado, la frase "Hola Mundo" cerca de la parte superior y la palabra "Hola" hacia la parte inferior, distribuidos a lo largo del espacio disponible en pantalla.



PRÁCTICA 2

practica2.dart:

practica2.dart

practica1 > lib > widget2 > practica2.dart > Disenio > build

```dart
import 'package:flutter/material.dart'; // Importa los componentes de interfaz gráfica de Flutter / Imports Flutter UI components

class Practica2 extends StatefulWidget { // Define el widget Practica2 con estado dinámico / Defines the Practica2 widget with dynamic state
  const Practica2({super.key}); // Constructor de la clase con clave de identificación / Class constructor with identification key
  @override // Sobrescribe el método de creación del estado / Overrides the state creation method
  State<StatefulWidget> createState() { // Crea el estado asociado al widget Practica2 / Creates state associated with Practica2 widget
    return Disenio(); // Retorna la clase de estado Disenio / Returns the Disenio state class
  }
}

class Disenio extends State<Practica2>{ // Clase de estado que controla el widget Practica2 / State class controlling the Practica2 widget
  @override // Sobrescribe el método de construcción de la interfaz / Overrides the UI build method
  Widget build(BuildContext context) { // Método para construir la estructura visual de la pantalla / Method to build the screen's visual structure
    return Scaffold( // Proporciona la estructura básica de la pantalla / Provides basic layout structure for the screen
      appBar: AppBar( // Configura la barra superior de la aplicación / Configures the top app bar
        title: Text("Practica 2", // Establece el título de la barra como "Practica 2" / Sets app bar title to "Practica 2"
        style: TextStyle( // Aplica estilo al texto del título / Applies style to title text
          color: Colors.white, // Define el color de texto en blanco / Sets text color to white
        ), // TextStyle
        ), // Text
        backgroundColor: const Color.fromARGB(255, 6, 86, 223), // Define el color de fondo de la barra en azul / Sets app bar background color to blue
      ), // AppBar

      body: Column( // Organiza los elementos hijos en una columna vertical / Layouts child elements in a vertical column
        mainAxisAlignment: MainAxisAlignment.center, // Centra los elementos verticalmente en la pantalla / Centers elements vertically on screen
        children: [ // Lista de widgets hijos dentro de la columna / List of child widgets inside the column
          Container( // Crea un contenedor para la primera caja de texto / Creates a container for the first text box
            color: Colors.black, // Establece el fondo del contenedor en negro / Sets container background color to black
            width: double.infinity, // Expande el contenedor a todo el ancho disponible / Expands container to full available width
            child: Text("Hola mundo", // Define un texto que muestra "Hola mundo" / Defines text displaying "Hola mundo"
            textAlign: TextAlign.center, // Centra el texto horizontalmente dentro del contenedor / Centers text horizontally inside container
            style: TextStyle( // Aplica estilo visual al texto / Applies visual style to text
              color: Colors.white, // Define el color del texto en blanco / Sets text color to white
              fontFamily: "Times New Roman", // Define la tipografía como Times New Roman / Sets font family to Times New Roman
              fontSize: 24, // Establece el tamaño de letra a 24 puntos / Sets font size to 24 points
            ), // TextStyle
            ), // Text
          ), // Container
          SizedBox( // Crea un espacio en blanco de separación / Creates an empty spacing box
            height: 10, // Define una altura de 10 píxeles de espacio / Sets height spacing to 10 pixels
          ), // SizedBox
          Container( // Crea un segundo contenedor con relleno interno / Creates a second container with internal padding
            padding: EdgeInsets.all(25), // Aplica un relleno interno de 25 píxeles en todos los lados / Applies 25 pixel padding on all sides
            color: const Color.fromARGB(255, 248, 24, 24), // Define el color de fondo en rojo / Sets background color to red
            width: double.infinity, // Expande el contenedor a todo el ancho disponible / Expands container to full available width
            child: Text("Hola", // Define un texto que muestra "Hola" / Defines text displaying "Hola"
            textAlign: TextAlign.center, // Centra el texto horizontalmente / Centers text horizontally
            style: TextStyle( // Aplica estilo visual al texto / Applies visual style to text
              color: Colors.white, // Define el color del texto en blanco / Sets text color to white
              fontFamily: "Times New Roman", // Define la tipografía como Times New Roman / Sets font family to Times New Roman
              fontSize: 24, // Establece el tamaño de letra a 24 puntos / Sets font size to 24 points
            ), // TextStyle
            ), // Text
          ), // Container
          SizedBox( // Crea un espacio vertical de separación / Creates vertical spacing box
            height: 10, // Define una altura de 10 píxeles de espacio / Sets height spacing to 10 pixels
          ), // SizedBox
          Container( // Crea un contenedor para ajustar el ancho del botón / Creates a container to adjust button width
            width: double.infinity, // Extiende el botón a todo el ancho de la pantalla / Extends button across full screen width
            child: ElevatedButton( // Crea un botón elevado interactivo / Creates an interactive elevated button
              onPressed: (){}, // Define la acción al presionar el botón (vacía) / Defines action on button press (empty)
              child: Text('Boton') // Asigna la etiqueta 'Boton' al botón / Sets the 'Boton' label on the button
              ), // ElevatedButton
          ), // Container
          SizedBox( // Crea un espacio vertical de separación / Creates vertical spacing box
            height: 10, // Define una altura de 10 píxeles de espacio / Sets height spacing to 10 pixels
          ), // SizedBox
          TextField( // Crea un campo de entrada de texto para el usuario / Creates a text input field for the user
            decoration: InputDecoration( // Define la decoración y estilo del campo de texto / Defines decoration and style for the text field
              labelText: 'Escribe el nombre', // Muestra una etiqueta flotante 'Escribe el nombre' / Displays floating label 'Escribe el nombre'
              border: OutlineInputBorder(), // Agrega un borde rectangular alrededor del campo / Adds a rectangular border around the field
            ), // InputDecoration
          ), // TextField
        ],
      ) // Column
    ); // Scaffold
  }
}
```



En esta segunda práctica, la pantalla del teléfono muestra en la parte superior una barra azul con el título "Practica 2" en texto blanco. En el centro del teléfono se organizan verticalmente cuatro elementos separados por un pequeño espacio. Primero, una barra negra que cubre todo el ancho con el texto "Hola mundo" centrado; debajo, una caja roja más gruesa (gracias al relleno interno) con el texto "Hola"; le sigue un botón interactivo a todo lo ancho con la leyenda "Boton"; y finalmente, un campo de texto interactivo delimitado por un borde rectangular con la etiqueta flotante "Escribe el nombre" para que el usuario pueda escribir.



PRÁCTICA 3

suma.dart:

```dart
import 'package:flutter/material.dart'; // Importa los componentes de interfaz gráfica de Flutter / Imports Flutter UI components

class Suma extends StatefulWidget{ // Define el widget Suma con estado dinámico / Defines the Suma widget with dynamic state
  @override // Sobrescribe el método de creación del estado / Overrides the state creation method
  State<StatefulWidget> createState() { // Crea el estado asociado al widget Suma / Creates state associated with Suma widget
    return Disenio(); // Retorna la clase de estado Disenio / Returns the Disenio state class
  }
}

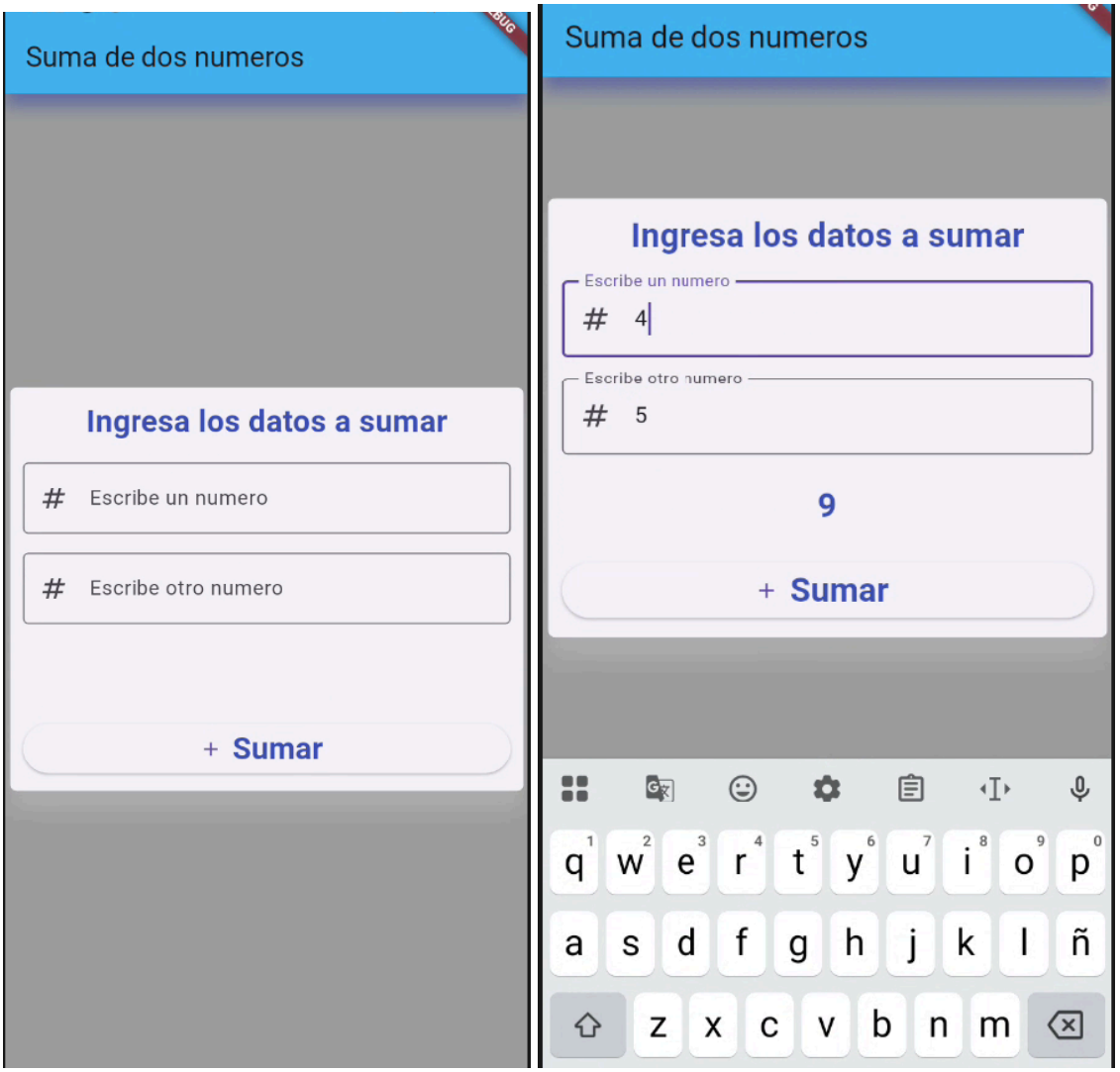
class Disenio extends State<Suma>{ // Clase de estado que controla la lógica e interfaz de Suma / State class controlling Suma logic and UI
  final TextEditingController n1 = TextEditingController(); // Controlador para capturar el primer número / Controller to capture the first number
  final TextEditingController n2 = TextEditingController(); // Controlador para capturar el segundo número / Controller to capture the second number

  String r = ""; // Variable de estado para almacenar el resultado / State variable to store the result

  @override // Sobrescribe el método de construcción de la interfaz / Overrides the UI build method
  Widget build(BuildContext context) { // Método para construir la estructura visual / Method to build the visual structure
    return Scaffold( // Proporciona la estructura básica de la pantalla / Provides basic layout structure for the screen
      backgroundColor: Colors.grey, // Define el color de fondo de la pantalla en gris / Sets screen background color to grey
      appBar: AppBar( // Configura la barra superior de la aplicación / Configures the top app bar
        title: Text("Suma de dos numeros"), // Define el título de la barra superior / Sets the top bar title
        backgroundColor: Colors.lightBlueAccent, // Establece el color de fondo en azul claro / Sets background color to light blue
        elevation: 10, // Define la sombra de elevación de la barra en 10 / Sets top bar shadow elevation to 10
        shadowColor: Colors.indigoAccent, // Establece el color de la sombra de la barra / Sets top bar shadow color
      ), // AppBar
      body: Center( // Centra el contenido en el medio de la pantalla / Centers content in the middle of the screen
        child: Card( // Crea una tarjeta elevada para agrupar elementos / Creates an elevated card to group elements
          shape: RoundedRectangleBorder( // Define la forma con bordes redondeados para la tarjeta / Defines rounded border shape for the card
            borderRadius: BorderRadius.circular(5) // Aplica un radio de curvatura de 5 píxeles / Applies 5 pixel corner radius
          ), // RoundedRectangleBorder
          elevation: 15, // Aplica una elevación de sombra de 15 píxeles a la tarjeta / Applies 15 pixel shadow elevation to card
          shadowColor: Colors.white, // Define el color de la sombra de la tarjeta en blanco / Sets card shadow color to white
          child: Padding( // Añade un margen interno alrededor del contenido / Adds internal padding around content
            padding:EdgeInsets.all(10), // Define 10 píxeles de relleno en todos los lados / Sets 10 pixels of padding on all sides
            child: Column( // Organiza los elementos hijos en una columna vertical / Layouts child elements in a vertical column
              mainAxisSize: MainAxisSize.min, // Ajusta la altura de la columna al tamaño de sus hijos / Fits column height to child size
              children: [ // Lista de widgets dentro de la columna / List of widgets inside the column
                Text("Ingresa los datos a sumar", // Muestra el título de instrucciones / Displays instruction title
                style: TextStyle( // Define el estilo visual del texto de instrucciones / Defines visual style for instruction text
                  fontSize: 24, // Establece el tamaño de letra en 24 puntos / Sets font size to 24 points
                  fontWeight: FontWeight.bold, // Aplica estilo de texto en negrita / Sets text style to bold
                  color: Colors.indigo, // Define el color del texto en índigo / Sets text color to indigo
                ), // TextStyle
                ), // Text
                SizedBox(height: 15,), // Crea un espacio vertical de 15 píxeles / Creates 15 pixel vertical spacing
                TextField( // Campo de texto para ingresar el primer número / Text field to input the first number
                  controller: n1, // Asigna el controlador n1 al campo de texto / Assigns n1 controller to text field
                  decoration: InputDecoration( // Define la decoración visual del campo de texto / Defines visual decoration for text field
                    labelText: "Escribe un numero", // Muestra la etiqueta descriptiva del campo / Displays descriptive field label
                    border: OutlineInputBorder(), // Agrega un borde rectangular alrededor del campo / Adds rectangular border around field
                    prefixIcon: Icon(Icons.numbers) // Agrega un ícono de números al inicio del campo / Adds numbers icon at field start
                  ), // InputDecoration
                ), // TextField
```

```dart
                SizedBox(height: 15,), // Crea un espacio vertical de 15 píxeles / Creates 15 pixel vertical spacing
                TextField( // Campo de texto para ingresar el segundo número / Text field to input the second number
                  controller: n2, // Asigna el controlador n2 al campo de texto / Assigns n2 controller to text field
                  decoration: InputDecoration( // Define la decoración visual del segundo campo / Defines visual decoration for second field
                    labelText: "Escribe otro numero", // Muestra la etiqueta del segundo campo / Displays second field label
                    border: OutlineInputBorder(), // Agrega un borde rectangular alrededor del campo / Adds rectangular border around field
                    prefixIcon: Icon(Icons.numbers) // Agrega un ícono de números al inicio del campo / Adds numbers icon at field start
                  ), // InputDecoration
                ), // TextField
                SizedBox(height: 20,), // Crea un espacio vertical de 20 píxeles / Creates 20 pixel vertical spacing
                Text(r, // Muestra el resultado almacenado en la variable r / Displays the result stored in variable r
                style: TextStyle( // Aplica estilo visual al texto del resultado / Applies visual style to result text
                  fontSize: 24, // Establece el tamaño de letra en 24 puntos / Sets font size to 24 points
                  fontWeight: FontWeight.bold, // Aplica estilo de texto en negrita / Sets text style to bold
                  color: Colors.indigo, // Define el color del texto en índigo / Sets text color to indigo
                ), // TextStyle
                ), // Text
                SizedBox(height: 20,), // Crea un espacio vertical de 20 píxeles / Creates 20 pixel vertical spacing
                SizedBox( // Contenedor para controlar el ancho del botón / Container to control button width
                  width: double.infinity, // Expande el botón a todo el ancho disponible / Expands button to full available width
                  child: ElevatedButton.icon( // Crea un botón elevado con ícono y texto / Creates an elevated button with icon and label
                    icon: Icon(Icons.add), // Define el ícono de suma para el botón / Sets addition icon for the button
                    onPressed: (){ // Función ejecutada al presionar el botón / Function executed when button is pressed
                      final int a = int.tryParse(n1.text) ?? 0; // Convierte el texto de n1 a entero o asigna 0 / Converts n1 text to int or assigns 0
                      final int b = int.tryParse(n2.text) ?? 0; // Convierte el texto de n2 a entero o asigna 0 / Converts n2 text to int or assigns 0

                      setState(() { // Actualiza el estado y redibuja la interfaz / Updates state and redraws UI
                        r = "${a + b}"; // Suma a y b, guardando el resultado en r / Adds a and b, saving result in r
                      });
                    },
                    label: Text("Sumar", // Asigna la etiqueta 'Sumar' al botón / Sets 'Sumar' label on the button
                    style: TextStyle( // Aplica estilo visual a la etiqueta del botón / Applies visual style to button label
                    fontSize: 24, // Establece el tamaño de letra en 24 puntos / Sets font size to 24 points
                    fontWeight: FontWeight.bold, // Aplica texto en negrita / Sets text to bold
                    color: Colors.indigo, // Define el color del texto en índigo / Sets text color to indigo
                    ), // TextStyle
                    ) // Text
                  ), // ElevatedButton.icon
                ), // SizedBox
              ],
            ), // Column
          ), // Padding
        ), // Card
      ), // Center
    ); // Scaffold
  }
}
```



En esta práctica se diseñó una calculadora interactiva para sumar dos números. En la pantalla del móvil se observa un fondo gris con una barra superior azul claro titulada "Suma de dos números". En el centro del teléfono se observa una tarjeta blanca elevada que contiene el título "Ingresa los datos a sumar" en texto azul índigo en negrita, seguido de dos campos de entrada con íconos para escribir cada número, un espacio central donde se muestra dinámicamente el resultado y, en la parte inferior, un botón a todo lo ancho con el ícono "+" y la etiqueta "Sumar" que, al presionarlo, calcula la suma de ambos valores y actualiza de inmediato el resultado en pantalla.

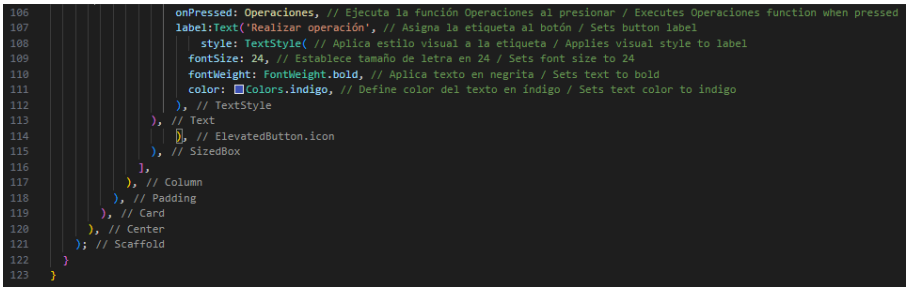


PRÁCTICA 4

practica4.dart:

```
practica1 > lib > operaciones > practica4.dart > Clases > build
1   import 'package:flutter/material.dart'; // Importa los componentes de interfaz gráfica de Flutter / Imports Flutter UI components
2
3   class Programa4 extends StatefulWidget { // Define el widget Programa4 con estado dinámico / Defines the Programa4 widget with dynamic state
4     @override // Sobrescribe el método de creación del estado / Overrides the state creation method
5     State<StatefulWidget> createState() { // Crea el estado asociado al widget Programa4 / Creates state associated with Programa4 widget
6       return Clases(); // Retorna la clase de estado Clases / Returns the Clases state class
7     }
8   }
9
10  class Clases extends State<Programa4>{ // Clase de estado que controla la lógica e interfaz / State class controlling logic and UI
11    final TextEditingController n1 = TextEditingController(); // Controlador para el primer número / Controller for the first number
12    final TextEditingController n2 = TextEditingController(); // Controlador para el segundo número / Controller for the second number
13
14    String r = "", oper=""; // Variables para almacenar el resultado y la operación / Variables to store result and operation
15
16    void Operaciones(){ // Función para calcular el resultado según la operación / Function to calculate result based on operation
17      final int a = int.tryParse(n1.text) ?? 0; // Convierte n1 a entero o usa 0 / Converts n1 to int or uses 0
18      final int b = int.tryParse(n2.text) ?? 0; // Convierte n2 a entero o usa 0 / Converts n2 to int or uses 0
19      setState(() { // Actualiza el estado de la aplicación / Updates application state
20        print("Oper: "+oper); // Imprime la operación elegida en consola / Prints selected operation in console
21        if (oper == "Suma"){ // Verifica si la operación es Suma / Checks if operation is Suma
22          r = "${a + b}"; // Calcula la suma / Calculates addition
23        }
24        if (oper == "Resta"){ // Verifica si la operación es Resta / Checks if operation is Resta
25          r = "${a - b}"; // Calcula la resta / Calculates subtraction
26        }
27        if (oper == "Multiplicación"){ // Verifica si la operación es Multiplicación / Checks if operation is Multiplication
28          r = "${a * b}"; // Calcula la multiplicación / Calculates multiplication
29        }
30        if (oper == "División"){ // Verifica si la operación es División / Checks if operation is Division
31          r = "${a / b}"; // Calcula la división / Calculates division
32        }
33        ScaffoldMessenger.of(context).showSnackBar( // Muestra un mensaje emergente en pantalla / Shows a floating snackbar message on screen
34          SnackBar(content: Text("El resultado: " +r)), // Define el texto del mensaje emergente con el resultado / Defines snackbar text with result
35        );
36      });
37    }
38
39    @override // Sobrescribe el método de construcción de la interfaz / Overrides UI build method
40    Widget build(BuildContext context) { // Método para construir la estructura visual / Method to build visual structure
41      return Scaffold( // Proporciona la estructura básica de la pantalla / Provides basic screen layout structure
42        backgroundColor: Colors.grey, // Define el color de fondo en gris / Sets background color to grey
43        appBar: AppBar( // Configura la barra superior / Configures the top bar
44          title: Text('Practica 4'), // Define el título "Practica 4" / Sets title to "Practica 4"
45          backgroundColor: const Color.fromARGB(255, 192, 214, 240), // Define el color de la barra en azul claro / Sets app bar color to light blue
46          elevation: 10, // Define la sombra de elevación / Sets shadow elevation
47          shadowColor: Colors.indigoAccent, // Define el color de la sombra / Sets shadow color
48        ), // AppBar
49        body: Center( // Centra el contenido en pantalla / Centers content on screen
50          child: Card( // Crea una tarjeta contenedor elevada / Creates an elevated container card
51            shape: RoundedRectangleBorder( // Define la forma con bordes redondeados / Defines rounded border shape
52              borderRadius: BorderRadius.circular(5), // Aplica radio de curvatura de 5 píxeles / Applies 5 pixel corner radius
53            ), // RoundedRectangleBorder
```

```
54            elevation: 15, // Aplica elevación de sombra a la tarjeta / Applies shadow elevation to card
55            shadowColor: Colors.white, // Define color de sombra de la tarjeta / Sets card shadow color
56            child: Padding( // Añade espacio interno alrededor del contenido / Adds padding around content
57              padding: EdgeInsets.all(20), // Aplica 20 píxeles de relleno / Applies 20 pixels of padding
58              child: Column( // Organiza los elementos en columna vertical / Layouts elements in vertical column
59                mainAxisSize: MainAxisSize.min, // Ajusta la altura al tamaño de los hijos / Fits height to child elements size
60                children: [ // Lista de widgets dentro de la columna / List of widgets inside the column
61                  Text('Ingresa los datos a sumar', // Muestra el título de instrucciones / Displays instruction title
62                  style: TextStyle( // Define el estilo visual del título / Defines visual style for title
63                    fontSize: 24, // Establece tamaño de letra en 24 / Sets font size to 24
64                    fontWeight: FontWeight.bold, // Aplica texto en negrita / Sets text to bold
65                    color: Colors.indigo, // Define color de texto en índigo / Sets text color to indigo
66                  ), // TextStyle
67                  ), // Text
68                  SizedBox(height: 15), // Espacio vertical de 15 píxeles / 15 pixel vertical spacing
69                  TextField( // Campo de texto para la primera entrada / Text field for first input
70                    controller: n1, // Asigna el controlador n1 / Assigns n1 controller
71                    decoration: InputDecoration( // Define la decoración del campo / Defines field decoration
72                      labelText: "Escribe un numero", // Etiqueta para el primer campo / Label for first field
73                      border: OutlineInputBorder(), // Agrega borde rectangular / Adds rectangular border
74                      prefixIcon: Icon(Icons.numbers) // Agrega ícono de números / Adds numbers icon
75                    ), // InputDecoration
76                  ), // TextField
77                  SizedBox(height: 15), // Espacio vertical de 15 píxeles / 15 pixel vertical spacing
78                  TextField( // Campo de texto para la segunda entrada / Text field for second input
79                    controller: n2, // Asigna el controlador n2 / Assigns n2 controller
80                    decoration: InputDecoration( // Define la decoración del campo / Defines field decoration
81                      labelText: "Escribe otro numero", // Etiqueta para el segundo campo / Label for second field
82                      border: OutlineInputBorder(), // Agrega borde rectangular / Adds rectangular border
83                      prefixIcon: Icon(Icons.numbers) // Agrega ícono de números / Adds numbers icon
84                    ), // InputDecoration
85                  ), // TextField
86                  SizedBox(height: 20,), // Espacio vertical de 20 píxeles / 20 pixel vertical spacing
87                  DropdownButtonFormField<String>( // Menú desplegable para seleccionar la operación / Dropdown field to select operation
88                    decoration: InputDecoration( // Define la decoración del desplegable / Defines dropdown decoration
89                      labelText: "Elige la operación", // Etiqueta del menú desplegable / Dropdown label
90                      border: OutlineInputBorder(), // Borde rectangular para el desplegable / Rectangular border for dropdown
91                    ), // InputDecoration
92                    items: [ // Lista de opciones del menú desplegable / List of dropdown options
93                      DropdownMenuItem(value: 'Suma', child:Text('Suma')), // Opción para sumar / Option for addition
94                      DropdownMenuItem(value: 'Resta', child:Text('Resta')), // Opción para restar / Option for subtraction
95                      DropdownMenuItem(value: 'Multiplicación', child:Text('Multiplicación')), // Opción para multiplicar / Option for multiplication
96                      DropdownMenuItem(value: 'División', child:Text('División')), // Opción para dividir / Option for division
97                    ],
98                    onChanged: (value) => oper = value!, // Guarda la opción seleccionada en oper / Saves selected option in oper
99                    validator: (v) => v == null ? "Elige una operacion": null, // Valida que se seleccione una opción / Validates option selection
100                 ), // DropdownButtonFormField
101                 SizedBox(height: 20,), // Espacio vertical de 20 píxeles / 20 pixel vertical spacing
102                 SizedBox( // Contenedor para controlar el ancho del botón / Container to control button width
103                   width: double.infinity, // Expande el botón a todo el ancho disponible / Expands button to full available width
104                   child: ElevatedButton.icon( // Crea un botón elevado con ícono / Creates an elevated button with icon
105                     icon: Icon(Icons.calculate), // Define el ícono de calculadora / Sets calculator icon
```



```
                    onPressed: Operaciones, // Ejecuta la función Operaciones al presionar / Executes Operaciones function when pressed
                    label:Text('Realizar operación', // Asigna la etiqueta al botón / Sets button label
                        style: TextStyle( // Aplica estilo visual a la etiqueta / Applies visual style to label
                      fontSize: 24, // Establece tamaño de letra en 24 / Sets font size to 24
                      fontWeight: FontWeight.bold, // Aplica texto en negrita / Sets text to bold
                      color: Colors.indigo, // Define color del texto en índigo / Sets text color to indigo
                    ), // TextStyle
                  ), // Text
                    ), // ElevatedButton.icon
                  ), // SizedBox
                ],
              ), // Column
            ), // Padding
          ), // Card
        ), // Center
      ); // Scaffold
    }
  }
```



En esta cuarta práctica se creó una calculadora con selector de operaciones matemáticas. En la pantalla del móvil se aprecia un fondo gris con una barra superior azul claro titulada "Practica 4". En el centro del teléfono destaca una tarjeta blanca elevada que contiene el título "Ingresa los datos a sumar" en azul índigo en negrita, dos campos de texto con íconos para escribir los números, un menú desplegable con borde para seleccionar la operación (Suma, Resta, Multiplicación o División) y, en la parte inferior, un botón a todo lo ancho con el ícono de calculadora y la etiqueta "Realizar operación" que, al presionarlo, calcula el resultado según la opción elegida y lo muestra mediante un mensaje emergente flotante (SnackBar) en la pantalla.



PRÁCTICA 5

main.dart:

```dart
import 'package:flutter/material.dart'; // Importa los componentes de diseño Material de Flutter / Imports Flutter Material design components
import 'package:practica5/widget/mostrarimagenes.dart'; // Importa el widget para mostrar imágenes / Imports the widget to display images

void main() { // Función principal de punto de entrada / Main entry point function
  runApp(const MyApp()); // Inicia la aplicación cargando el widget MyApp / Starts the application loading the MyApp widget
}

class MyApp extends StatelessWidget { // Define la clase principal MyApp como widget sin estado / Defines the main MyApp class as a stateless widget
  const MyApp({super.key}); // Constructor constante con clave de identificación / Constant constructor with identification key

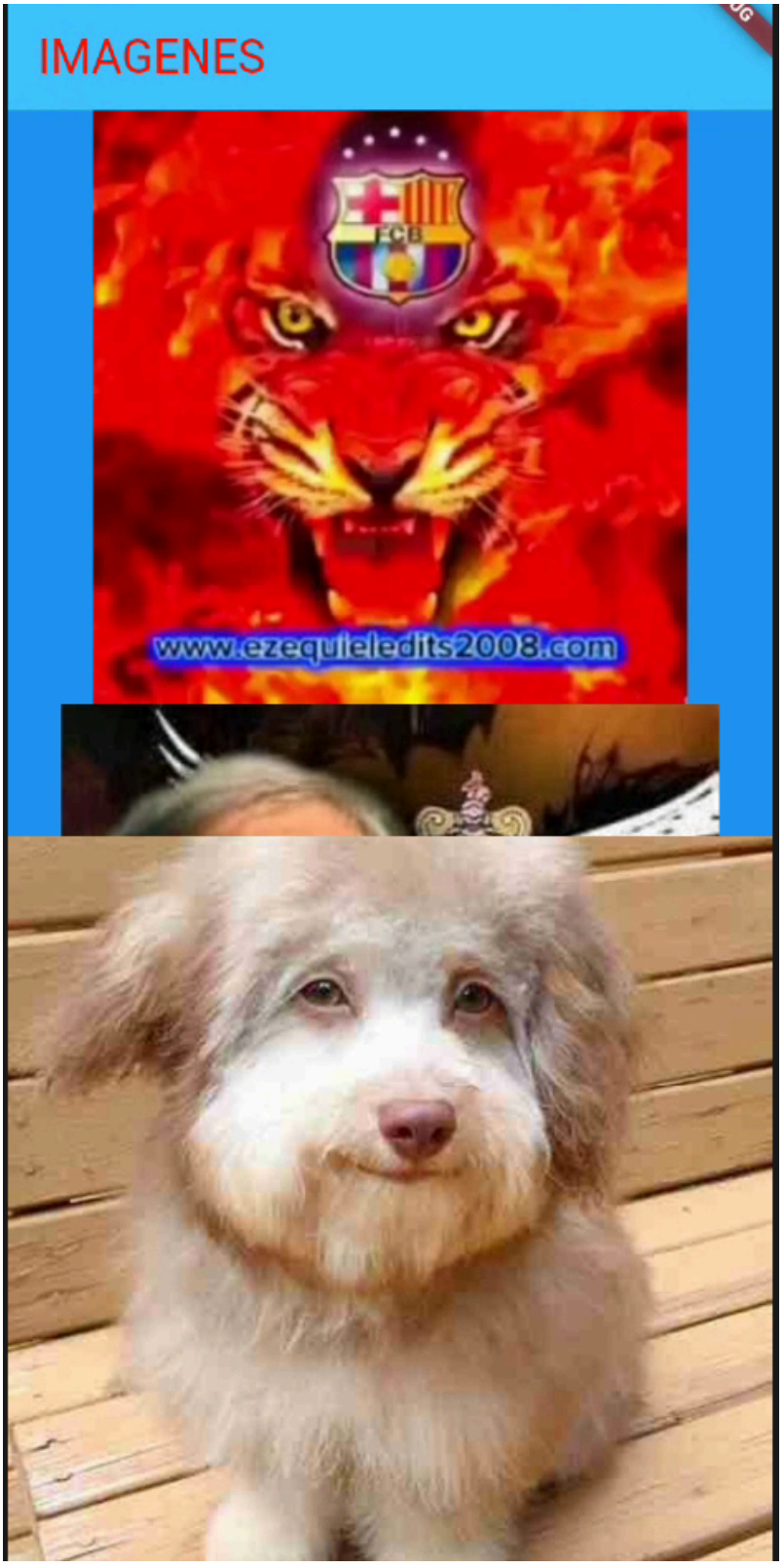
  @override // Sobrescribe el método de construcción / Overrides the build method
  Widget build(BuildContext context) { // Método que construye la interfaz gráfica / Method that builds the user interface
    return MaterialApp( // Retorna el contenedor de aplicaciones Material Design / Returns the Material Design app wrapper
      home: MostrarImagenes(), // Define el widget MostrarImagenes como pantalla principal / Sets the MostrarImagenes widget as the main screen
    ); // MaterialApp
  }
}
```

mostrarimagenes.dart:

```dart
import 'package:flutter/material.dart'; // Importa los componentes de interfaz gráfica de Flutter / Imports Flutter UI components

class MostrarImagenes extends StatefulWidget{ // Define el widget MostrarImagenes con estado dinámico / Defines the MostrarImagenes widget with dynamic state
  @override // Sobrescribe el método de creación del estado / Overrides state creation method
  State<StatefulWidget> createState() { // Crea el estado asociado al widget MostrarImagenes / Creates state associated with MostrarImagenes widget
    return imagenes(); // Retorna la clase de estado imagenes / Returns the imagenes state class
  }
}

class imagenes extends State<MostrarImagenes>{ // Clase de estado que controla la interfaz de MostrarImagenes / State class controlling MostrarImagenes UI
  @override // Sobrescribe el método de construcción de la interfaz / Overrides UI build method
  Widget build(Object context) { // Método para construir la estructura visual / Method to build visual structure
    return Scaffold( // Proporciona la estructura básica de la pantalla / Provides basic screen layout structure
      backgroundColor: Colors.blue, // Define el color de fondo de la pantalla en azul / Sets screen background color to blue
      appBar: AppBar( // Configura la barra superior de la aplicación / Configures the top app bar
        title: Text('IMAGENES', // Define el título 'IMAGENES' en la barra / Sets title 'IMAGENES' in app bar
        style: TextStyle( // Aplica estilos visuales al texto del título / Applies visual styles to title text
          color: const Color.fromARGB(255, 255, 17, 0), // Define el color del texto en rojo / Sets text color to red
          fontSize: 25, // Establece el tamaño de letra en 25 puntos / Sets font size to 25 points
        ), // TextStyle
        ), // Text
        backgroundColor: Colors.lightBlueAccent, // Define el fondo de la barra en azul claro / Sets app bar background to light blue
      ), // AppBar
      body: Column( // Organiza el contenido en una columna vertical principal / Layouts content in a main vertical column
        children: [ // Lista de widgets hijos dentro de la columna principal / List of child widgets inside main column
          Expanded( // Expande la primera sección para ocupar la mitad del espacio / Expands first section to take half the space
            child: Container( // Contenedor para el primer bloque de imágenes / Container for the first block of images
              width: double.infinity, // Ocupa todo el ancho disponible / Takes full available width
              height: double.infinity, // Ocupa toda la altura asignada por el Expanded / Takes full height assigned by Expanded
              child: Center( // Centra el contenido internamente / Centers content internally
                child: SingleChildScrollView( // Permite el desplazamiento vertical del contenido / Enables vertical scrolling for content
                  child: Column( // Organiza las imágenes verticalmente / Layouts images vertically
                    children: [ // Lista de imágenes del primer bloque / List of images in the first block
                      Image.asset( // Carga una imagen desde los activos locales / Loads an image from local assets
                        "assets/imagen1.jpg", // Ruta de la primera imagen / Path of the first image
                      ), // Image.asset
                      Image.asset( // Carga la segunda imagen local / Loads second local image
                        "assets/imagen2.jpg", // Ruta de la segunda imagen / Path of the second image
                      ), // Image.asset
                      Image.asset( // Carga la tercera imagen local / Loads third local image
                        "assets/imagen3.jpg", // Ruta de la tercera imagen / Path of the third image
                      ) // Image.asset
                    ],
                  ), // Column
                ), // SingleChildScrollView
              ), // Center
            ), // Container
          ), // Expanded
          Expanded( // Expande la segunda sección para ocupar la otra mitad del espacio / Expands second section to take the other half of space
            child: Container( // Contenedor para el segundo bloque de imágenes / Container for the second block of images
              width: double.infinity, // Ocupa todo el ancho disponible / Takes full available width
              height: double.infinity, // Ocupa toda la altura asignada por el Expanded / Takes full height assigned by Expanded
              child: Center( // Centra el contenido internamente / Centers content internally
                child: SingleChildScrollView( // Habilita desplazamiento vertical para el segundo bloque / Enables vertical scrolling for second block
                  child: Column( // Organiza el segundo grupo de imágenes / Layouts second group of images
                    children: [ // Lista de imágenes del segundo bloque / List of images in the second block
                      Image.asset( // Carga la primera imagen del segundo grupo / Loads first image of second group
                        "assets/imagen1.jpg", // Ruta de la imagen 1 / Path of image 1
                      ), // Image.asset
                      Image.asset( // Carga la segunda imagen del segundo grupo / Loads second image of second group
                        "assets/imagen2.jpg", // Ruta de la imagen 2 / Path of image 2
                      ), // Image.asset
                      Image.asset( // Carga la tercera imagen del segundo grupo / Loads third image of second group
                        "assets/imagen3.jpg", // Ruta de la imagen 3 / Path of image 3
                      ) // Image.asset
                    ],
                  ), // Column
                ), // SingleChildScrollView
              ), // Center
            ), // Container
          ), // Expanded
        ],
      ), // Column
    ); // Scaffold
  }
}
```



En esta quinta práctica se diseñó una interfaz para visualizar varias imágenes divididas en pantalla. En el móvil se observa un fondo azul con una barra superior azul claro que muestra el título "IMAGENES" en letras rojas de gran tamaño. El área principal del teléfono se divide verticalmente en dos secciones de igual tamaño (una superior y una inferior); cada mitad cuenta con su propio contenedor con desplazamiento independiente, lo que permite recorrer hacia abajo una serie de tres imágenes (imagen1.jpg, imagen2.jpg e imagen3.jpg) tanto en el bloque de arriba como en el de abajo.



PRÁCTICA 6

main.dart:

```dart
import 'package:flutter/material.dart'; // Importa los componentes de diseño Material de Flutter / Imports Flutter Material design components
import 'package:practica6/widget/login.dart'; // Importa el widget de la pantalla de inicio de sesión / Imports the login screen widget

void main() { // Función principal de punto de entrada / Main entry point function
  runApp(const MyApp()); // Inicia la aplicación cargando el widget MyApp / Starts the application loading the MyApp widget
}

class MyApp extends StatelessWidget { // Define la raíz de la aplicación como un widget sin estado / Defines the app root as a stateless widget
  const MyApp({super.key}); // Constructor constante con clave de identificación / Constant constructor with identification key

  @override // Sobrescribe el método de construcción / Overrides the build method
  Widget build(BuildContext context) { // Método que construye la interfaz gráfica / Method that builds the user interface
    return MaterialApp( // Retorna la estructura principal basada en Material Design / Returns the main structure based on Material Design
      home: Login(), // Define el widget Login como la pantalla principal / Sets the Login widget as the main screen
    ); // MaterialApp
  }
}
```

login.dart:

```dart
import 'package:flutter/material.dart'; // Importa los componentes de interfaz gráfica de Flutter / Imports Flutter UI components
import 'package:practica6/widget/acceso.dart'; // Importa el widget de la pantalla de acceso / Imports the access screen widget

class Login extends StatefulWidget{ // Define el widget Login con estado dinámico / Defines the Login widget with dynamic state
  @override // Sobrescribe el método de creación del estado / Overrides state creation method
  State<StatefulWidget> createState() { // Crea el estado asociado al widget Login / Creates state associated with Login widget
    return Clase(); // Retorna la clase de estado Clase / Returns the Clase state class
  }
}

class Clase extends State<Login>{ // Clase de estado que controla la lógica de inicio de sesión / State class controlling login logic
  TextEditingController usuario = new TextEditingController(); // Controlador para la caja de texto del usuario / Controller for user text field
  TextEditingController password = new TextEditingController(); // Controlador para la caja de texto de la contraseña / Controller for password text field

  void validar(){ // Función que valida las credenciales ingresadas / Function that validates entered credentials
    String u = usuario.text; // Guarda el valor ingresado en usuario / Stores entered username value
    String p = password.text; // Guarda el valor ingresado en contraseña / Stores entered password value

    setState(() { // Actualiza el estado de la aplicación / Updates application state
      if(u == "admin" && p == "12345"){ // Comprueba si el usuario y contraseña son correctos / Checks if username and password are correct
        Navigator.pushReplacement( // Navega a la nueva pantalla reemplazando la actual / Navigates to new screen replacing current one
            context,
            MaterialPageRoute( // Define la ruta de transición entre pantallas / Defines transition route between screens
              builder: (context) => Acceso( // Construye el widget de la pantalla Acceso / Builds the Acceso screen widget
                dato : u, // Envía el nombre de usuario como parámetro / Passes username as parameter
                datop : p // Envía la contraseña como parámetro / Passes password as parameter
              ) // Acceso
            ) // MaterialPageRoute
        );
      }else{ // Acción si las credenciales son incorrectas / Action if credentials are incorrect
        Alertas("Error", "Datos incorrectos"); // Muestra una alerta con el mensaje de error / Displays an alert with error message
        usuario.clear(); // Limpia el contenido del campo de usuario / Clears username field content
        password.clear(); // Limpia el contenido del campo de contraseña / Clears password field content
      }
    });
  }

  void Alertas(String titulo,String mensaje){ // Función para mostrar cuadros de diálogo de alerta / Function to show alert dialog boxes

    showDialog( // Muestra un diálogo modal en pantalla / Displays a modal dialog on screen
      context: context,
      builder: (context){
        return AlertDialog( // Crea la ventana de alerta modal / Creates modal alert window
          title: Text(titulo, // Establece el texto del título de la alerta / Sets alert title text
            style: TextStyle( // Define el estilo del título / Defines title style
              fontSize: 22, // Tamaño de fuente 22 puntos / Font size 22 points
              color: Colors.blue // Color de fuente azul / Blue font color
            ), // TextStyle
          ), // Text
          content: Text(mensaje, // Establece el mensaje interno de la alerta / Sets alert content message
            style: TextStyle( // Define el estilo del mensaje / Defines message style
              fontSize: 14, // Tamaño de fuente 14 puntos / Font size 14 points
              color: Colors.blue // Color de fuente azul / Blue font color
            ), // TextStyle
          ), // Text
          actions: [ // Lista de botones de acción del diálogo / List of dialog action buttons
            TextButton( // Botón plano para cerrar el diálogo / Flat button to close dialog
              onPressed: (){ // Acción al presionar el botón / Action when button is pressed
                Navigator.of(context).pop(); // Cierra el cuadro de diálogo modal / Closes modal dialog box
              },
              child: Text("Aceptar") // Texto del botón de cierre / Close button text
            ) // TextButton
          ],
        ); // AlertDialog
      }
    );
  }

  @override // Sobrescribe el método de construcción de interfaz / Overrides UI build method
  Widget build(BuildContext context) { // Método para construir la estructura visual / Method to build visual structure
    return Scaffold( // Estructura básica de la pantalla / Basic layout structure of the screen
      backgroundColor: const Color.fromARGB(255, 215, 215, 215), // Define fondo gris claro / Sets light grey background
      appBar: AppBar( // Configura la barra superior / Configures top app bar
        title: Text('Login'), // Título de la barra de navegación / Navigation bar title
        backgroundColor: Colors.cyanAccent, // Fondo azul cian para la barra / Cyan background for app bar
      ), // AppBar
      body: Center( // Centra el contenido en pantalla / Centers content on screen
        child: Padding( // Agrega relleno alrededor de la tarjeta / Adds padding around card
          padding: EdgeInsets.all(15), // Relleno de 15 píxeles en todos los lados / 15 pixel padding on all sides
          child: Card( // Tarjeta contenedora elevada / Elevated container card
            elevation: 8, // Define elevación de sombra de 8 píxeles / Sets 8 pixel shadow elevation
            shadowColor: Colors.black, // Color de sombra negro / Black shadow color
            shape: RoundedRectangleBorder( // Define bordes redondeados para la tarjeta / Defines rounded borders for card
              borderRadius: BorderRadius.circular(15) // Radio de curvatura de 15 píxeles / 15 pixel corner radius
            ), // RoundedRectangleBorder
            child: Padding( // Agrega espacio interno a la tarjeta / Adds internal padding to card
              padding: const EdgeInsets.all(8.0), // Relleno interno de 8 píxeles / 8 pixel internal padding
              child: Column( // Organiza los elementos verticalmente / Layouts elements vertically
                mainAxisSize: MainAxisSize.min, // Ajusta altura al tamaño de los hijos / Fits height to child elements size
                children: [ // Lista de elementos dentro de la tarjeta / List of elements inside card
                  Text('Ingresa los datos', // Título descriptivo superior / Upper descriptive title
                  style: TextStyle( // Estilo del título / Title style
                    fontSize: 22, // Tamaño de letra 22 puntos / Font size 22 points
                    color: Colors.blue // Color azul / Blue color
                  ), // TextStyle
                  ), // Text
                  SizedBox(height: 10), // Espacio vertical de 10 píxeles / 10 pixel vertical spacing
                  Container( // Contenedor circular para la imagen de perfil / Circular container for profile image
                    width: 120, // Ancho de 120 píxeles / 120 pixel width
                    height: 120, // Altura de 120 píxeles / 120 pixel height
                    decoration: BoxDecoration( // Define la decoración circular del contenedor / Defines circular decoration of container
                      shape: BoxShape.circle, // Forma circular / Circular shape
                      color: Colors.white // Fondo blanco / White background
                    ), // BoxDecoration
                    child: ClipOval( // Recorta el contenido en forma ovalada/circular / Clips content into oval/circular shape
                      child: Image.asset("assets/img.png"), // Carga la imagen del perfil / Loads profile image
```



```
                    ), // ClipOval
                  ), // Container
                  SizedBox(height: 10), // Espacio vertical de 10 píxeles / 10 pixel vertical spacing
                  Container( // Contenedor para la entrada de usuario / Container for user input
                    width: double.infinity, // Ocupa todo el ancho / Takes full width
                    child: TextField( // Campo de texto de usuario / User text field
                      controller: usuario, // Asigna controlador de usuario / Assigns user controller
                      decoration: InputDecoration( // Decoración del campo / Field decoration
                        labelText: "Escribe el Usuario", // Etiqueta principal / Main label
                        hintText: "Ingresa el Usuario...", // Texto de sugerencia / Placeholder text
                        border: OutlineInputBorder( // Borde redondeado / Rounded border
                          borderRadius: BorderRadius.circular(10) // Radio de curvatura de 10 píxeles / 10 pixel corner radius
                        ) // OutlineInputBorder
                      ), // InputDecoration
                    ), // TextField
                  ), // Container
                  SizedBox(height: 10), // Espacio vertical de 10 píxeles / 10 pixel vertical spacing
                  Container( // Contenedor para la entrada de contraseña / Container for password input
                    width: double.infinity, // Ocupa todo el ancho / Takes full width
                    child: TextField( // Campo de texto de contraseña / Password text field
                      controller: password, // Asigna controlador de contraseña / Assigns password controller
                      decoration: InputDecoration( // Decoración del campo / Field decoration
                        labelText: "Escribe el Password", // Etiqueta principal / Main label
                        hintText: "Ingresa el Password...", // Texto de sugerencia / Placeholder text
                        border: OutlineInputBorder( // Borde redondeado / Rounded border
                          borderRadius: BorderRadius.circular(10) // Radio de curvatura de 10 píxeles / 10 pixel corner radius
                        ) // OutlineInputBorder
                      ), // InputDecoration
                    ), // TextField
                  ), // Container
                  SizedBox(height: 10), // Espacio vertical de 10 píxeles / 10 pixel vertical spacing
                  Container( // Contenedor para el botón de confirmación / Container for confirmation button
                    width: double.infinity, // Ocupa todo el ancho / Takes full width
                    child: ElevatedButton.icon( // Botón elevado con ícono / Elevated button with icon
                      onPressed: validar, // Ejecuta la función de validación al presionar / Executes validation function when pressed
                      label: Text('Aceptar', // Etiqueta del botón / Button label
                      style: TextStyle( // Estilo del texto del botón / Button text style
                        fontSize: 16, // Tamaño de letra 16 puntos / Font size 16 points
                        color: Colors.blue // Color azul / Blue color
                        ), // TextStyle
                      ) // Text
                    ), // ElevatedButton.icon
                  ) // Container
                ],
              ), // Column
            ), // Padding
          ), // Card
          ), // Padding
        ), // Center
      ); // Scaffold
    }
}
```

acceso.dart:

```
import 'package:flutter/material.dart'; // Importa los componentes de interfaz gráfica de Flutter / Imports Flutter UI components

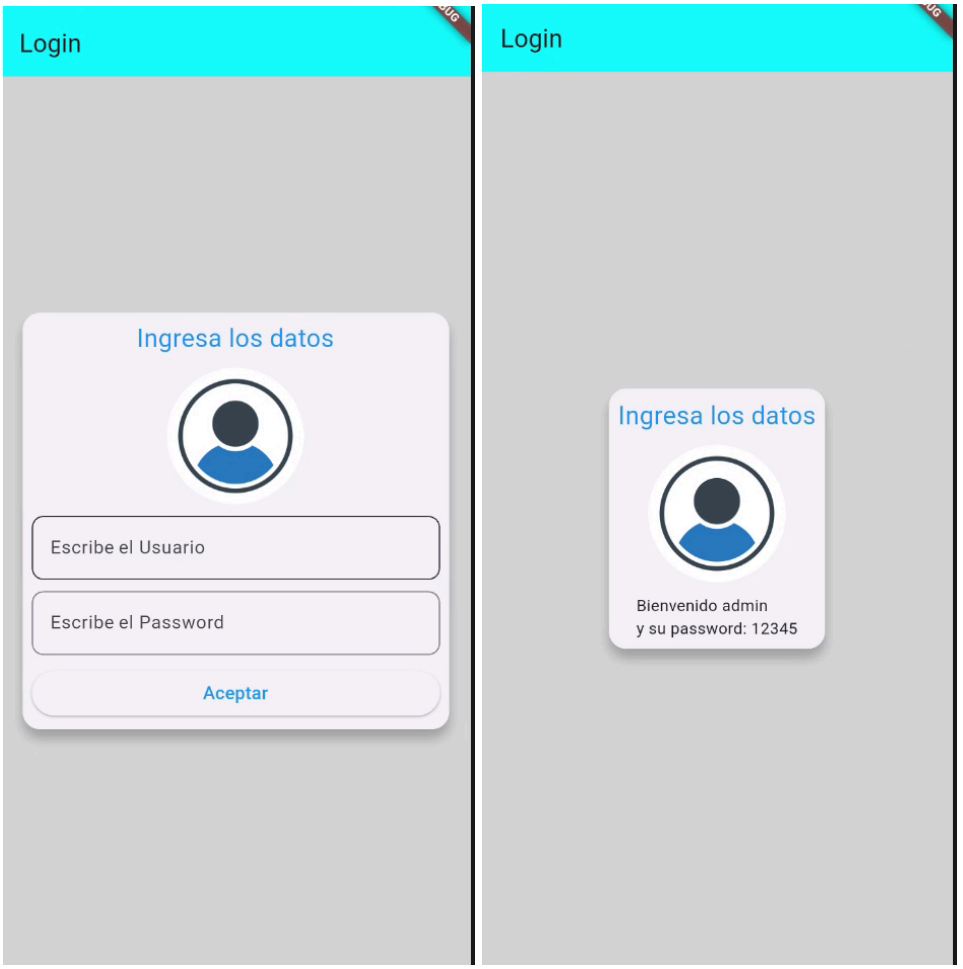
class Acceso extends StatefulWidget{ // Define el widget Acceso con estado dinámico / Defines the Acceso widget with dynamic state
  final String dato, datop; // Variables finales para recibir el usuario y la contraseña / Final variables to receive username and password

  const Acceso({super.key, required this.dato, required this.datop}); // Constructor constante que requiere los parámetros dato y datop / Constant constructor requiring parameters dato and datop
  @override // Sobrescribe el método de creación del estado / Overrides state creation method
  State<StatefulWidget> createState() { // Crea el estado asociado al widget Acceso / Creates state associated with Acceso widget
    return Clase(); // Retorna la clase de estado Clase / Returns the Clase state class
  }
}

class Clase extends State<Acceso>{ // Clase de estado que controla la pantalla de Acceso / State class controlling Acceso screen
  String d = "", p = ""; // Variables locales para almacenar usuario y contraseña / Local variables to store username and password

@override // Sobrescribe el método del ciclo de vida initState / Overrides initState lifecycle method
  void initState(){ // Se ejecuta al inicio de la inicialización de la clase / Executes at the beginning of class initialization
    super.initState(); // Llama a la inicialización de la clase padre / Calls parent class initialization
    d = widget.dato; // Asigna el valor del usuario recibido desde el widget / Assigns username value received from widget
    p = widget.datop; // Asigna el valor de la contraseña recibida desde el widget / Assigns password value received from widget
  }

  @override // Sobrescribe el método de construcción de interfaz / Overrides UI build method
  Widget build(BuildContext context) { // Método para construir la estructura visual / Method to build visual structure
    return Scaffold( // Proporciona la estructura básica de la pantalla / Provides basic screen layout structure
      backgroundColor: const Color.fromARGB(255, 215, 215, 215), // Define fondo gris claro / Sets light grey background
      appBar: AppBar( // Configura la barra superior / Configures top app bar
        title: Text('Login'), // Título de la barra de navegación / Navigation bar title
        backgroundColor: Colors.cyanAccent, // Fondo azul cian para la barra / Cyan background for app bar
      ), // AppBar
      body: Center( // Centra el contenido en pantalla / Centers content on screen
        child: Padding( // Agrega relleno alrededor del contenedor / Adds padding around container
          padding: EdgeInsets.all(15), // Relleno de 15 píxeles en todos los lados / 15 pixel padding on all sides
          child: Card( // Tarjeta contenedora con sombra / Container card with shadow
            elevation: 8, // Define elevación de sombra de 8 píxeles / Sets 8 pixel shadow elevation
            shadowColor: Colors.black, // Color de sombra negro / Black shadow color
            shape: RoundedRectangleBorder( // Define bordes redondeados para la tarjeta / Defines rounded borders for card
              borderRadius: BorderRadius.circular(15) // Radio de curvatura de 15 píxeles / 15 pixel corner radius
            ), // RoundedRectangleBorder
            child: Padding( // Agrega espacio interno a la tarjeta / Adds internal padding to card
              padding: const EdgeInsets.all(8.0), // Relleno interno de 8 píxeles / 8 pixel internal padding
              child: Column( // Organiza los elementos verticalmente / Layouts elements vertically
                mainAxisSize: MainAxisSize.min, // Ajusta altura al tamaño de los hijos / Fits height to child elements size
                children: [ // Lista de elementos dentro de la tarjeta / List of elements inside card
                  Text('Ingresa los datos', // Título informativo superior / Upper informative title
                  style: TextStyle( // Estilo del título / Title style
                    fontSize: 22, // Tamaño de letra 22 puntos / Font size 22 points
                    color: Colors.blue // Color azul / Blue color
                  ), // TextStyle
                  ), // Text
                  SizedBox(height: 10), // Espacio vertical de 10 píxeles / 10 pixel vertical spacing
                  Container( // Contenedor circular para la imagen de perfil / Circular container for profile image
                    width: 120, // Ancho de 120 píxeles / 120 pixel width
                    height: 120, // Altura de 120 píxeles / 120 pixel height
                    decoration: BoxDecoration( // Define la decoración circular del contenedor / Defines circular decoration of container
                      shape: BoxShape.circle, // Forma circular / Circular shape
                      color: Colors.white // Fondo blanco / White background
                    ), // BoxDecoration
                    child: ClipOval( // Recorta la imagen en forma circular / Clips image to circle shape
                      child: Image.asset("assets/img.png"), // Carga la imagen del perfil desde activos / Loads profile image from assets
                    ), // ClipOval
                  ), // Container
                  SizedBox(height: 10), // Espacio vertical de 10 píxeles / 10 pixel vertical spacing
                  Text("Bienvenido $d \ny su password: $p") // Muestra el usuario y la contraseña recibidos / Displays received username and password
                ],
              ), // Column
            ), // Padding
          ), // Card
          ), // Padding
        ), // Center
      ); // Scaffold
    }
}
```



En esta sexta práctica se diseñó una interfaz de inicio de sesión con validación de datos y navegación con transferencia de datos entre pantallas en Flutter. En la pantalla del móvil se observa un fondo gris claro con una barra superior azul cian que muestra el título 'Login'. En el centro se despliega una tarjeta blanca elevada con bordes redondeados que contiene el mensaje 'Ingresa los datos' en letras azules, una imagen de perfil circular, dos campos de texto con bordes contorneados para escribir el usuario y la contraseña, y un botón 'Aceptar'. Al presionar el botón, el sistema verifica los datos ingresados, y si son incorrectas, muestra un cuadro de diálogo de alerta con el mensaje de error y limpia las casillas de texto, pero si son correctas (admin y 12345), realiza la transición mediante un reemplazo de pantalla hacia la vista de 'Acceso', enviando los datos introducidos para mostrarlos en una tarjeta final de bienvenida con la confirmación del usuario y su contraseña.



UNIDAD 2
PRÁCTICA 7

aplicacion.dart:

```dart
import 'package:flutter/material.dart'; // Importa el paquete de Material UI de Flutter / Imports Flutter Material UI package
import 'package:practica7/widget/numero.dart'; // Importa el widget personalizado Numero / Imports custom Numero widget

class Aplicacion extends StatefulWidget {
  @override
  State<StatefulWidget> createState() {
    return Clase(); // Retorna la clase de estado asociada / Returns associated state class
  }
}

class Clase extends State<Aplicacion>{
  TextEditingController caja = new TextEditingController(); // Controlador para gestionar el campo de texto / Controller to manage text field
  String texto = ""; // Variable para almacenar mensajes de estado / Variable to store status messages

  void validar (){ // Método para validar si la caja de texto tiene contenido / Method to validate if text box has content
    String c = caja.text; // Obtiene el valor actual del campo de texto / Gets current value from text field
    setState(() {
      if(c == ""){
        texto = "La caja de texto esta vacia"; // Mensaje cuando está vacío / Message when empty
      }else{
        texto = "La caja de texto No esta vacia"; // Mensaje cuando tiene texto / Message when it has text
        alertas(); // Llama a la función que muestra una alerta / Calls function to show an alert
      }
    });
  }

  void alertas(){ // Método para mostrar un cuadro de diálogo de alerta / Method to display an alert dialog box
    showDialog(
      context: context,
      builder: (context){
        return AlertDialog(
          title: Text("Alerta"), // Título de la ventana de alerta / Alert dialog title
          content: Text(caja.text), // Muestra el texto ingresado dentro de la alerta / Displays entered text inside alert
          actions: [
            TextButton(
              onPressed: (){ // Acción al presionar el botón de la alerta / Action when alert button is pressed
                String d = caja.text; // Extrae el texto ingresado / Extracts entered text
                if(int.tryParse(d) != null){ // Comprueba si el texto es un número entero válido / Checks if text is a valid integer
                  Navigator.pushReplacement(
                    context,
                    MaterialPageRoute(
                      builder: ((context) => Numero(
                        dato : d // Envía el dato numérico al siguiente widget / Passes numeric data to the next widget
                      )) // Numero // MaterialPageRoute
                    );
                  caja.clear(); // Limpia la caja de texto / Clears the text field
                }else{
                  Navigator.of(context).pop(); // Cierra el cuadro de diálogo / Closes the dialog box
                }
              },
              child: Text("Aceptar") // Texto del botón de confirmación / Confirmation button text
            ) // TextButton
          ],
        ); // AlertDialog
      }
    );
  }

  @override
  Widget build(BuildContext context) {
    return Scaffold(
      backgroundColor: const Color.fromARGB(255, 201, 254, 140), // Color de fondo principal de la pantalla / Main screen background color
      appBar: AppBar(
        backgroundColor: const Color.fromARGB(255, 120, 222, 123), // Color de fondo de la barra superior / AppBar background color
        title: Text("Practica 7",
        style: TextStyle(
          color: Colors.white, // Color del texto del título / Title text color
          fontSize: 25 // Tamaño de fuente del título / Title font size
        ), // TextStyle
        ), // Text
      ), // AppBar
      body: Column(
        children: [
          Expanded(
            child: Center(
              child: Padding(
                padding: EdgeInsets.all(20),
                child: Card(
                  elevation: 8, // Elevación de la sombra de la tarjeta / Card shadow elevation
                  shadowColor: Colors.black, // Color de la sombra de la tarjeta / Card shadow color
                  shape: RoundedRectangleBorder(
                    borderRadius: BorderRadius.circular(8) // Bordes redondeados de la tarjeta / Card rounded corners
                  ), // RoundedRectangleBorder
                  child: Padding(
                    padding: EdgeInsets.all(30),
                    child: SingleChildScrollView(
                      child: Column(
                        children: [
                          SizedBox(height: 50), // Espacio vertical fijo / Fixed vertical spacing
                          Container(
                            width: double.infinity,
                            child: TextField(
                              controller: caja, // Vincula el controlador de texto / Binds text controller
                              decoration: InputDecoration(
                                labelText: "Escribe...", // Texto flotante indicador en el campo / Input floating label text
                                border: OutlineInputBorder(
                                  borderRadius: BorderRadius.circular(8) // Borde redondeado del campo de texto / Text field rounded border
                                ) // OutlineInputBorder
                              ), // InputDecoration
                            ), // TextField
                          ), // Container
                          SizedBox(height: 40), // Espacio vertical fijo / Fixed vertical spacing
                          Container(
                            width: double.infinity,
                            child: ElevatedButton.icon(
                              onPressed: validar, // Llama al método de validación al hacer clic / Calls validation method on click
                              label: Text("Aceptar") // Texto descriptivo del botón / Button label text
                            ), // ElevatedButton.icon
                          ) // Container
                        ],
                      ), // Column
                    ), // SingleChildScrollView
                  ), // Padding
                ), // Card
              ), // Padding
            ), // Center
          ), // Expanded
          Expanded(
            child: Center(
              child: Padding(
                padding: EdgeInsets.all(20),
                child: Card(
                  elevation: 8, // Elevación de la tarjeta inferior / Lower card shadow elevation
                  shadowColor: Colors.black, // Color de sombra de la tarjeta inferior / Lower card shadow color
                  shape: RoundedRectangleBorder(
                    borderRadius: BorderRadius.circular(8) // Bordes redondeados de la tarjeta / Card rounded corners
                  ), // RoundedRectangleBorder
                  child: Padding(
                    padding:  EdgeInsets.all(30.0),
                    child: Container(
                      width: double.infinity,
                      height: double.infinity,
                      child: Center(
                        child: Text(texto, // Muestra el mensaje de texto dinámico / Displays dynamic text message
                        style: TextStyle(
                          fontSize: 34, // Tamaño de la fuente del mensaje / Message font size
                          color: Colors.blue, // Color del texto del mensaje / Message text color
                          fontWeight: FontWeight.bold, // Aplica estilo negrita / Applies bold style
                          fontFamily: "Comic Sans" // Tipo de fuente tipográfica / Font family style
                        ) // TextStyle
                        ), // Text
                      ), // Center
                    ), // Container
                  ), // Padding
                ), // Card
              ), // Padding
            ), // Center
          ), // Expanded
```

numero.dart:

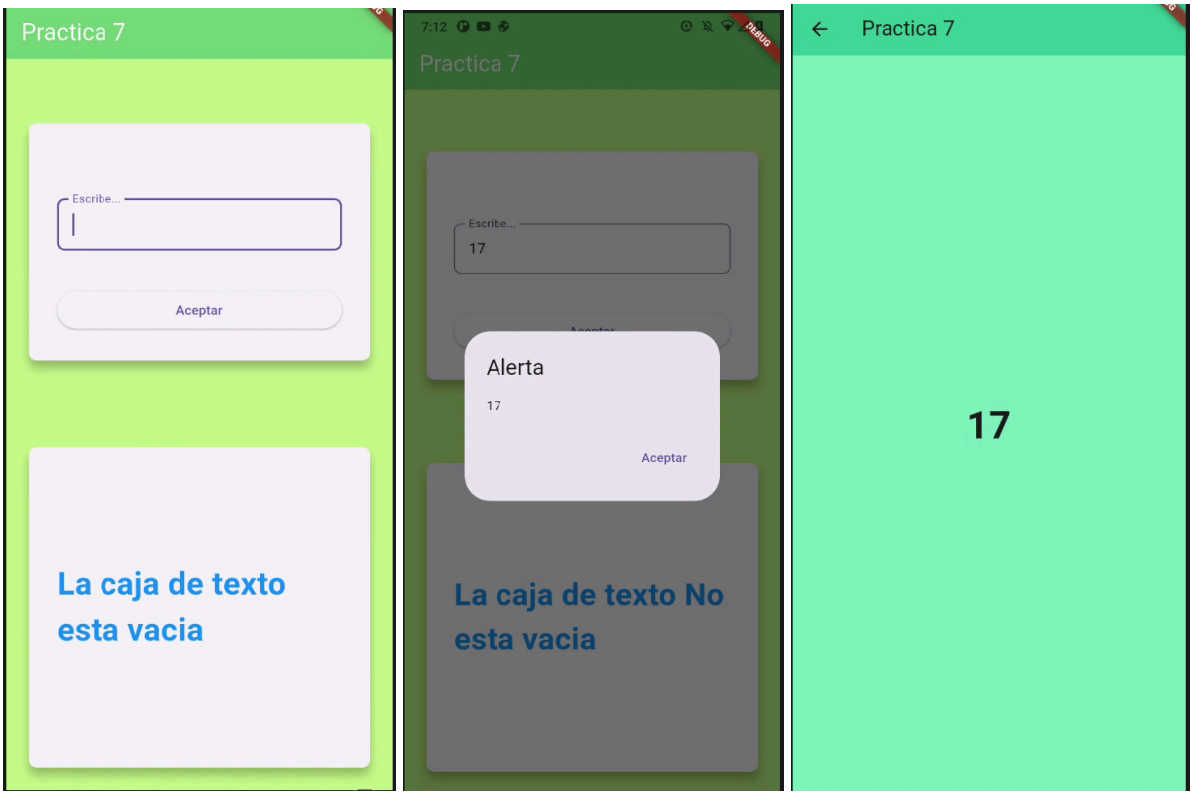
```dart
import 'package:flutter/material.dart'; // Importa el paquete Material UI de Flutter / Imports Flutter Material UI package

class Numero extends StatefulWidget{ // Define un StatefulWidget que recibe datos / Defines a StatefulWidget that receives dat
  final String dato; // Variable final para almacenar el dato recibido / Final variable to store received data

  const Numero({super.key, required this.dato}); // Constructor con parámetro obligatorio / Constructor with required paramete
  @override
  State<StatefulWidget> createState() {
    return Clase(); // Retorna la clase de estado / Returns the state class
  }
}

class Clase extends State<Numero>{ // Clase de estado para Numero / State class for Numero
  String d = ""; // Variable para mostrar el dato en pantalla / Variable to display data on screen
@override
  void initState(){
    super.initState(); // Inicializa el ciclo de vida del estado / Initializes state lifecycle
    d = widget.dato; // Asigna el dato recibido del widget padre / Assigns data received from parent widget
  }

  @override
  Widget build(BuildContext context) {
    return Scaffold( // Estructura base de la pantalla / Base screen layout
      backgroundColor: const Color.fromARGB(255, 130, 246, 190), // Color de fondo de la pantalla / Screen background color
      appBar: AppBar( // Barra superior de la aplicación / Application top app bar
        title: Text("Practica 7"), // Título de la AppBar / AppBar title
        backgroundColor: const Color.fromARGB(255, 69, 219, 154), // Color de fondo de la AppBar / AppBar background color
      ), // AppBar
      body: Center( // Centra el contenido en pantalla / Centers content on screen
        child: Text(d, // Muestra el dato recibido en formato de texto / Displays received data as text
        style: TextStyle( // Estilos para el texto del dato / Styles for data text
          fontSize: 40, // Tamaño de fuente grande / Large font size
          fontWeight: FontWeight.bold // Texto en negrita / Bold text
        ), // TextStyle
        ), // Text
      ), // Center
    ); // Scaffold
  }
}
```


Practica 7
Escribe...
Aceptar
La caja de texto esta vacia
7:12
DEBUG
Practica 7
Escribe...
17
Alerta
17
Aceptar
La caja de texto No esta vacia
Practica 7
17

PRÁCTICA 8

lista1.dart:

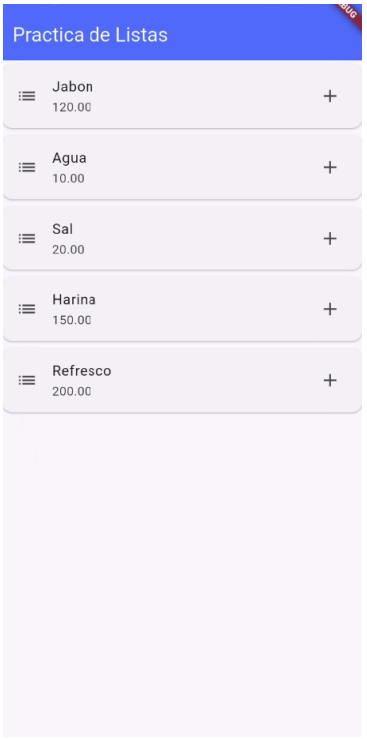
```dart
import 'package:flutter/material.dart'; // Importa el paquete Material UI de Flutter / Imports Flutter Material UI package

class Lista1 extends StatefulWidget{ // Define un StatefulWidget para la lista / Defines a StatefulWidget for the list
  @override
  State<StatefulWidget> createState() {
    return Clase2(); // Retorna la instancia de estado asociada / Returns associated state instance
  }
}

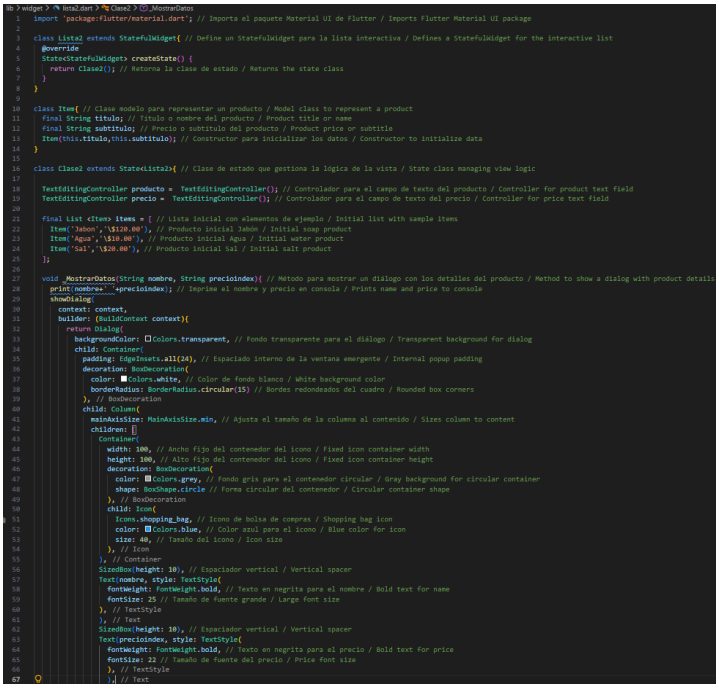
class Item{ // Clase modelo para representar un elemento / Model class to represent an item
  final String titulo; // Título o nombre del artículo / Title or item name
  final String subtitulo; // Subtítulo o precio del artículo / Subtitle or item price
  Item(this.titulo,this.subtitulo); // Constructor para inicializar el ítem / Constructor to initialize the item
}

class Clase2 extends State<Lista1>{ // Clase de estado que gestiona los elementos de la lista / State class managing list items
  final List <Item> items = [ // Lista con los datos de ejemplo / List with sample data
    Item('Jabon','120.00'), // Elemento Jabón / Soap item
    Item('Agua','10.00'), // Elemento Agua / Water item
    Item('Sal','20.00'), // Elemento Sal / Salt item
    Item('Harina','150.00'), // Elemento Harina / Flour item
    Item('Refresco','200.00'), // Elemento Refresco / Soda item
  ];

  @override
  Widget build(BuildContext context) {
    return Scaffold( // Estructura visual básica de la pantalla / Basic visual screen layout
      appBar: AppBar( // Barra superior de navegación / Top navigation app bar
        backgroundColor: Colors.indigoAccent, // Color de fondo de la AppBar / AppBar background color
        title: Text( // Widget de texto para el título / Text widget for the title
          "Practica de Listas", // Texto del título / Title text
          style: TextStyle( // Estilos para el texto / Styles for text
            color: Colors.white // Color blanco para el texto / White color for text
          ), // TextStyle
        ), // Text
      ), // AppBar
      body: ListView.builder( // Constructor eficiente para listas dinámicas / Efficient builder for dynamic lists
        itemCount: items.length, // Define la cantidad total de elementos / Defines total number of items
        itemBuilder: (context, index) =>Card( // Crea una tarjeta por cada elemento de la lista / Creates a card for each list item
          child: ListTile( // Estructura de elemento de lista / List tile layout structure
            leading: Icon(Icons.list), // Icono a la izquierda del elemento / Icon on the left of the item
            /*title: Text("Lista"),
            subtitle: Text("Subtitulo de la lista"),*/
            title: Text(items[index].titulo), // Muestra el título del ítem actual / Displays current item title
            subtitle: Text(items[index].subtitulo), // Muestra el subtítulo del ítem actual / Displays current item subtitle
            trailing: Icon(Icons.add), // Icono a la derecha del elemento / Icon on the right of the item
          ), // ListTile
        ) // Card
        ), // ListView.builder
      ); // Scaffold
    }
}
```



lista2.dart:



```dart
import 'package:flutter/material.dart'; // Importa el paquete Material UI de Flutter / Imports Flutter Material UI package

class Lista2 extends StatefulWidget{ // Define un StatefulWidget para la lista interactiva / Defines a StatefulWidget for the interactive list
  @override
  State<StatefulWidget> createState() {
    return Clase2(); // Retorna la clase de estado / Returns the state class
  }
}

class Item{ // Clase modelo para representar un producto / Model class to represent a product
  final String titulo; // Título o nombre del producto / Product title or name
  final String subtitulo; // Precio o subtítulo del producto / Product price or subtitle
  Item(this.titulo,this.subtitulo); // Constructor para inicializar los datos / Constructor to initialize data
}

class Clase2 extends State<Lista2>{ // Clase de estado que gestiona la lógica de la vista / State class managing view logic

  TextEditingController producto = TextEditingController(); // Controlador para el campo de texto del producto / Controller for product text field
  TextEditingController precio = TextEditingController(); // Controlador para el campo de texto del precio / Controller for price text field

  final List <Item> items = [ // Lista inicial con elementos de ejemplo / Initial list with sample items
    Item('Jabon',"$120.00"), // Producto inicial Jabón / Initial soap product
    Item('Agua',"$10.00"), // Producto inicial Agua / Initial water product
    Item('Sal',"$20.00"), // Producto inicial Sal / Initial salt product
  ];

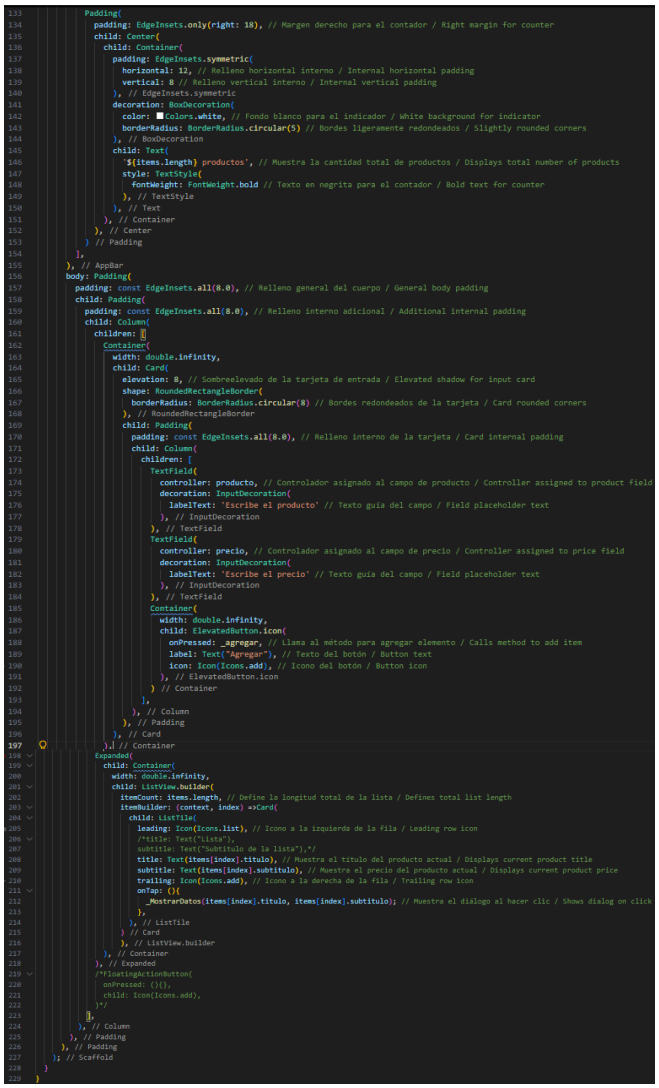
  void _MostrarDatos(String nombre, String precioindex){ // Método para mostrar un diálogo con los detalles del producto / Method to show a dialog with product details
    print(nombre+' '+precioindex); // Imprime el nombre y precio en consola / Prints name and price to console
    showDialog(
      context: context,
      builder: (BuildContext context){
        return Dialog(
          backgroundColor: Colors.transparent, // Fondo transparente para el diálogo / Transparent background for dialog
          child: Container(
            padding: EdgeInsets.all(24), // Espaciado interno de la ventana emergente / Internal popup padding
            decoration: BoxDecoration(
              color: Colors.white, // Color de fondo blanco / White background color
              borderRadius: BorderRadius.circular(15) // Bordes redondeados del cuadro / Rounded box corners
            ), // BoxDecoration
            child: Column(
              mainAxisSize: MainAxisSize.min, // Ajusta el tamaño de la columna al contenido / Sizes column to content
              children: [
                Container(
                  width: 100, // Ancho fijo del contenedor del icono / Fixed icon container width
                  height: 100, // Alto fijo del contenedor del icono / Fixed icon container height
                  decoration: BoxDecoration(
                    color: Colors.grey, // Fondo gris para el contenedor circular / Gray background for circular container
                    shape: BoxShape.circle // Forma circular del contenedor / Circular container shape
                  ), // BoxDecoration
                  child: Icon(
                    Icons.shopping_bag, // Icono de bolsa de compras / Shopping bag icon
                    color: Colors.blue, // Color azul para el icono / Blue color for icon
                    size: 40, // Tamaño del icono / Icon size
                  ), // Icon
                ), // Container
                SizedBox(height: 10), // Espaciador vertical / Vertical spacer
                Text(nombre, style: TextStyle(
                  fontWeight: FontWeight.bold, // Texto en negrita para el nombre / Bold text for name
                  fontSize: 25 // Tamaño de fuente grande / Large font size
                ), // TextStyle
                ), // Text
                SizedBox(height: 10), // Espaciador vertical / Vertical spacer
                Text(precioindex, style: TextStyle(
                  fontWeight: FontWeight.bold, // Texto en negrita para el precio / Bold text for price
                  fontSize: 22 // Tamaño de fuente del precio / Price font size
                ), // TextStyle
                ), // Text
                SizedBox(height: 10), // Espaciador vertical / Vertical spacer
                Container(
                  child: SizedBox(
                    width: double.infinity, // Ancho máximo disponible / Maximum available width
                    child: ElevatedButton(
                      onPressed: (){
                        Navigator.pop(context); // Cierra la ventana de diálogo / Closes the dialog window
                      },
                      child: Text("Cerrar",
                      style: TextStyle(
                        fontWeight: FontWeight.bold, // Texto en negrita para el botón / Bold text for button
                        fontSize: 24 // Tamaño de fuente del botón / Button font size
                      ), // TextStyle
                      ) // Text
                    ), // ElevatedButton
                  ), // SizedBox
                ) // Container
              ],
            ), // Column
          ); // Dialog
        }
    );

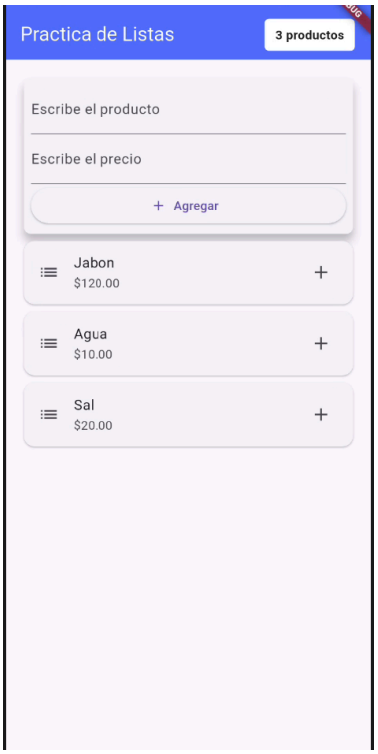
  }

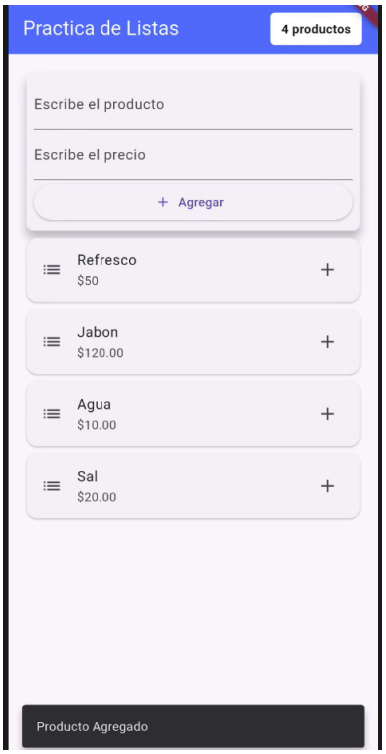
  void _agregar(){ // Método para registrar un nuevo producto / Method to register a new product
    String p = producto.text; // Obtiene el texto escrito en el campo de producto / Gets text written in product field
    String pr = precio.text; // Obtiene el texto escrito en el campo de precio / Gets text written in price field

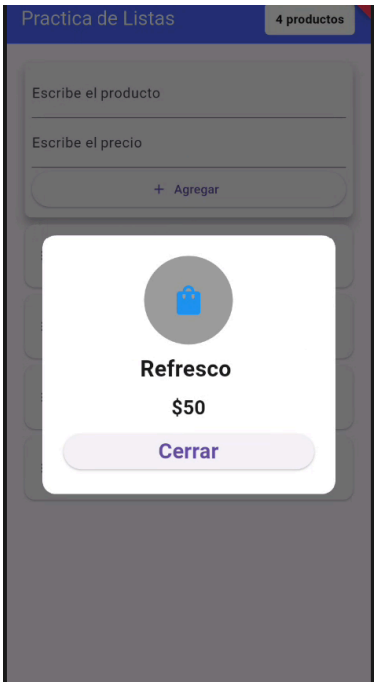
    if(p.isNotEmpty && pr.isNotEmpty){ // Verifica que ambos campos tengan contenido / Verifies both fields have content
      setState(() {
        items.insert(
          0, Item(p, '\$$pr') // Inserta el nuevo producto al inicio de la lista / Inserts new product at the start of the list
        );
      });
      producto.clear(); // Limpia la caja de texto del producto / Clears product text field
      precio.clear(); // Limpia la caja de texto del precio / Clears price text field
      FocusScope.of(context).unfocus(); // Oculta el teclado / Hides virtual keyboard
      ScaffoldMessenger.of(context).showSnackBar(
        SnackBar(content: Text('Producto Agregado'), // Mensaje emergente de éxito / Success toast message
        behavior: SnackBarBehavior.floating // Comportamiento flotante del SnackBar / Floating SnackBar behavior
        ) // SnackBar
      );
    }else{
      ScaffoldMessenger.of(context).showSnackBar(
        SnackBar(content: Text('Completa los datos'), // Mensaje de advertencia si faltan datos / Warning message if data is missing
        behavior: SnackBarBehavior.floating // Comportamiento flotante del SnackBar / Floating SnackBar behavior
        ) // SnackBar
      );
    }
  }

  @override
  Widget build(BuildContext context) {
    return Scaffold(
      appBar: AppBar(
        backgroundColor: Colors.indigoAccent, // Color de fondo de la barra superior / AppBar background color
        title: Text(
          "Practica de Listas",
          style: TextStyle(
            color: Colors.white // Color de texto blanco / White text color
          ), // TextStyle
        ), // Text
        actions: [
```




Practica de Listas
3 productos
Escribe el producto
Escribe el precio
+ Agregar
Jabon
$120.00
Agua
$10.00
Sal
$20.00


Practica de Listas
4 productos
Escribe el producto
Escribe el precio
+ Agregar
Refresco
$50
Jabon
$120.00
Agua
$10.00
Sal
$20.00
Producto Agregado


Practica de Listas
4 productos
Escribe el producto
Escribe el precio
+ Agregar
Refresco
$50
Cerrar

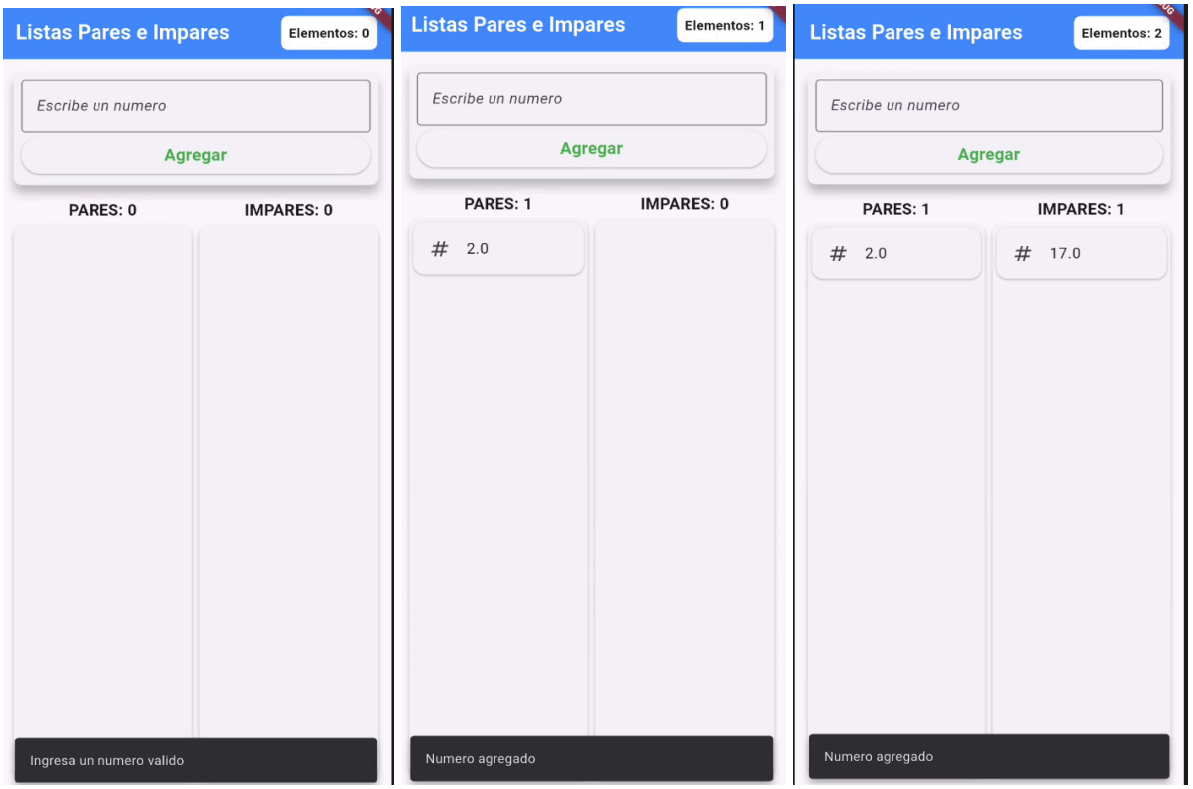
PRÁCTICA 9

listas.dart:





```dart
                        itemCount: impares.length, // Número de elementos en la lista de impares / Number of items in odd list
                        itemBuilder: (context, index) =>Card(
                          child: ListTile(
                            leading: Icon(Icons.numbers), // Icono indicador numérico / Numeric indicator icon
                            title: Text(impares[index]), // Muestra el número impar actual / Displays current odd number
                          ), // ListTile
                        ) // Card
                      ), // ListView.builder
                    ), // Card
                  ) // Expanded
                ],
              ), // Row
            ) // Container
          ) // Expanded
        ],
      ), // Column
    ), // Center
  ) // Padding
); // Scaffold
  }
}
```


Listas Pares e Impares
Elementos: 0
Escribe un numero
Agregar
PARES: 0
IMPARES: 0
Ingresa un numero valido
Listas Pares e Impares
Elementos: 1
Escribe un numero
Agregar
PARES: 1
IMPARES: 0
2.0
Numero agregado
Listas Pares e Impares
Elementos: 2
Escribe un numero
Agregar
PARES: 1
IMPARES: 1
2.0
17.0
Numero agregado

PRÁCTICA 10

ventana1.dart:

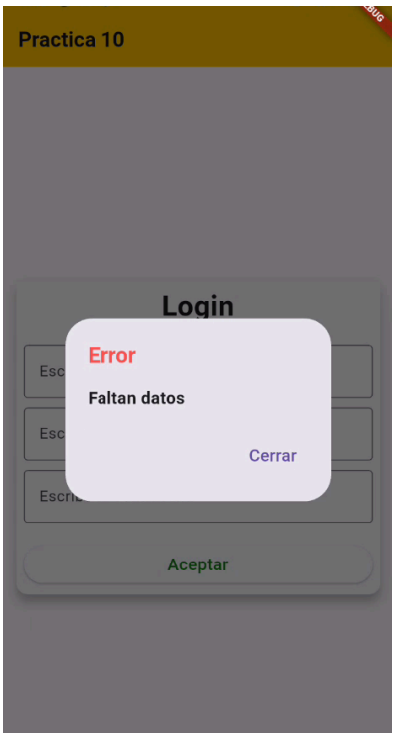




ventana2.dart:

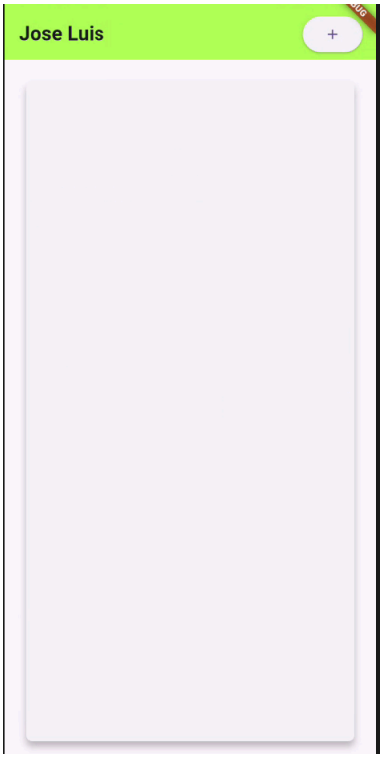




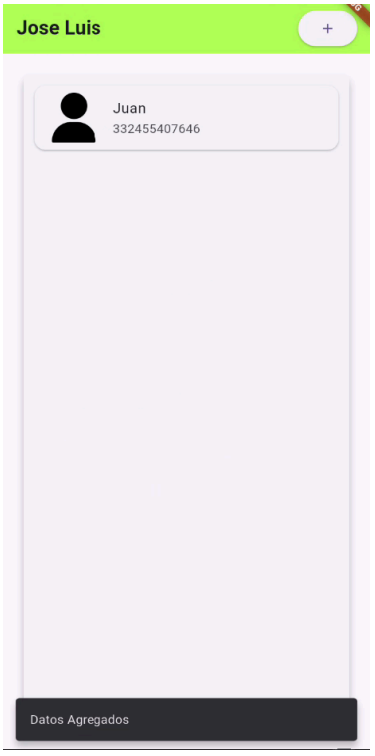

Practica 10
Login
Error
Faltan datos
Cerrar
Aceptar


Practica 10
Login
Error
El nombre no es valido
Cerrar
Aceptar


Login
Error
La contraseña no es valida
Cerrar
Aceptar


Jose Luis


Jose Luis
Agregar
Escribe el nombre
Juan
Escribe el telefono
332455407646
Aceptar


Jose Luis
Juan
332455407646
Datos Agregados

PRÁCTICA 11

datos.dart:



```dart
import 'package:flutter/material.dart'; // Importa el paquete Material UI de Flutter / Imports Flutter Material UI package
import 'package:practica11/widget/lista.dart'; // Importa el widget de lista personalizado / Imports custom list widget

class Datos extends StatefulWidget { // Define un StatefulWidget para capturar datos / Defines a StatefulWidget to capture data
  const Datos({super.key});
  @override
  State<StatefulWidget> createState() {
    return _DatosState(); // Retorna la clase de estado / Returns the state class
  }
}

class item{ // Clase modelo para representar el producto / Model class to represent the product
  final String producto; // Nombre del producto / Product name
  final String descripcion; // Descripción del producto / Product description
  final String precio; // Precio del producto / Product price
  item(this.producto,this.descripcion,this.precio); // Constructor para inicializar el item / Constructor to initialize the item
}

class _DatosState extends State<Datos>{ // Clase de estado que gestiona los controladores y la lista / State class managing controllers and list
  TextEditingController pro = TextEditingController(); // Controlador para el campo de producto / Controller for product field
  TextEditingController des = TextEditingController(); // Controlador para el campo de descripción / Controller for description field
  TextEditingController pre = TextEditingController(); // Controlador para el campo de precio / Controller for price field

  List <item> items = []; // Lista para almacenar los elementos agregados / List to store added items

  void _agregar(){ // Método para validar y agregar un elemento a la lista / Method to validate and add an item to the list
    setState(() {
      String p = pro.text; // Obtiene el texto del producto / Gets product text
      String d = des.text; // Obtiene el texto de la descripcion / Gets description text
      String pr = pre.text; // Obtiene el texto del precio / Gets price text
      if(p.isNotEmpty && d.isNotEmpty && pr.isNotEmpty){ // Verifica que ningún campo esté vacío / Verifies no field is empty
        items.add(item(p, d, pr)); // Añade el nuevo item a la lista / Adds new item to the list
        pro.clear(); // Limpia el campo de producto / Clears product field
        des.clear(); // Limpia el campo de descripción / Clears description field
        pre.clear(); // Limpia el campo de precio / Clears price field
        ScaffoldMessenger.of(context).showSnackBar(
          SnackBar(content: Text("Datos agregados a la lista")) // Mensaje flotante de éxito / Success toast message
        );
      }else{
        ScaffoldMessenger.of(context).showSnackBar(
          SnackBar(content: Text("Faltan Datos")) // Mensaje de advertencia si faltan datos / Warning message if data is missing
        );
      }
    });
  }

  void _enviar() async{ // Método asíncrono para navegar a la vista de lista / Async method to navigate to list view
    Navigator.push(
      context,
      MaterialPageRoute(
        builder: (context) => Lista(
          items : items // Pasa la lista de elementos al siguiente widget / Passes items list to the next widget
        ) // Lista
      ) // MaterialPageRoute
    );
    setState(() {}); // Actualiza el estado al regresar / Updates state upon return
  }

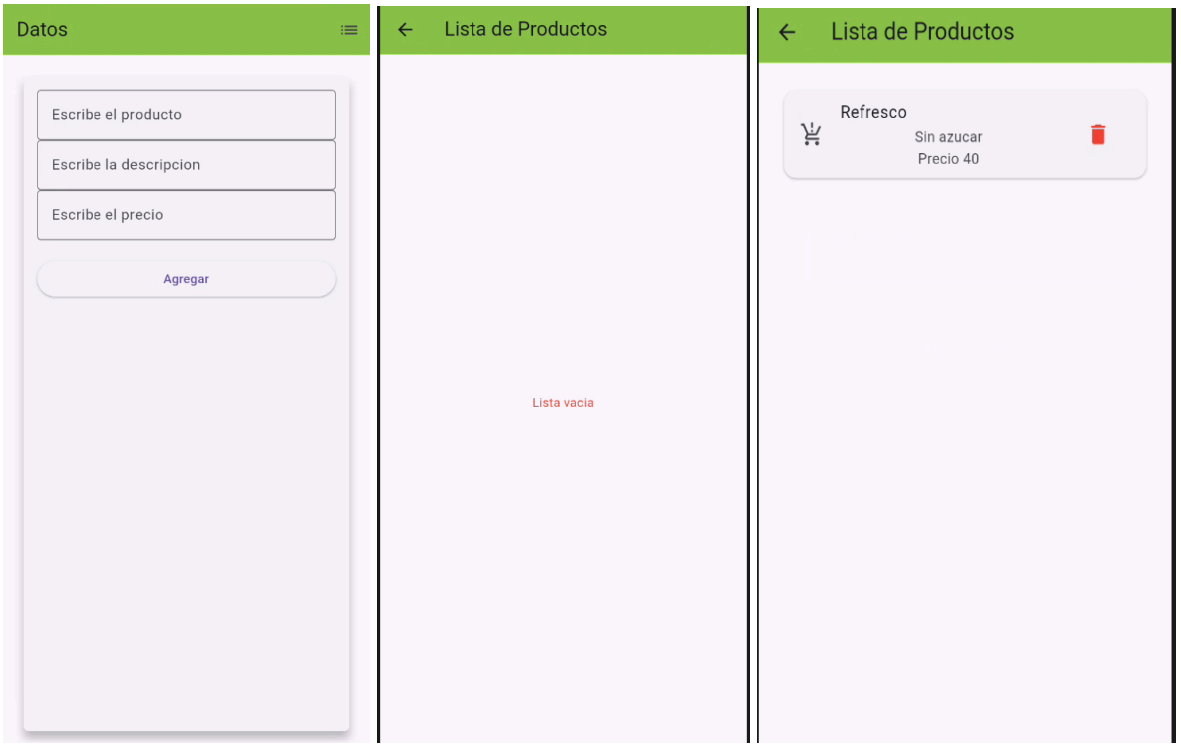
  @override
  Widget build(BuildContext context) {
    return Scaffold(
      appBar: AppBar(
        title: Text("Datos"), // Título de la AppBar / AppBar title
        backgroundColor: Colors.lightGreen, // Color de fondo verde claro / Light green background color
        actions: [
          IconButton(
            onPressed: _enviar, // Llama al método para enviar los datos a la otra vista / Calls method to send data to other view
            icon: Icon(Icons.list) // Icono de lista en la barra superior / List icon in top bar
          ) // IconButton
        ],
      ), // AppBar
      body: Center( // Centra el contenido en pantalla / Centers content on screen
        child: Padding(
          padding: EdgeInsets.all(20), // Relleno general / General padding
          child: Card(
            elevation: 8, // Sombreelevado de la tarjeta / Elevated card shadow
            shape: RoundedRectangleBorder(
              borderRadius: BorderRadius.circular(5) // Bordes redondeados de la tarjeta / Card rounded corners
            ), // RoundedRectangleBorder
            child: Padding(
              padding: const EdgeInsets.all(15), // Relleno interno de la tarjeta / Card internal padding
              child: Column(
                mainAxisSize: MainAxisSize.max, // Ocupa el máximo espacio disponible / Takes maximum available space
                children: [
                  TextField(
                    controller: pro, // Controlador asignado al campo de producto / Controller assigned to product field
                    decoration: InputDecoration(
                      labelText: "Escribe el producto", // Texto flotante del campo / Field floating label
                      border: OutlineInputBorder() // Borde delineado del campo / Outlined field border
                    ), // InputDecoration
                  ), // TextField
                  TextField(
                    controller: des, // Controlador asignado al campo de descripción / Controller assigned to description field
                    decoration: InputDecoration(
                      labelText: "Escribe la descripcion", // Texto flotante del campo / Field floating label
                      border: OutlineInputBorder() // Borde delineado del campo / Outlined field border
                    ), // InputDecoration
                  ), // TextField
                  TextField(
                    controller: pre, // Controlador asignado al campo de precio / Controller assigned to price field
                    decoration: InputDecoration(
                      labelText: "Escribe el precio", // Texto flotante del campo / Field floating label
                      border: OutlineInputBorder() // Borde delineado del campo / Outlined field border
                    ), // InputDecoration
                  ), // TextField
                  SizedBox(height: 20), // Espacio vertical de separación / Vertical separation space
                  SizedBox(
                    width: double.infinity, // Ancho máximo del botón / Maximum button width
                    child: ElevatedButton.icon(
                      onPressed: _agregar, // Llama al método para agregar los datos / Calls method to add data
                      label: Text("Agregar") // Texto del botón / Button text
                    ), // ElevatedButton.icon
                  ) // SizedBox
                ],
              ), // Column
            ), // Padding
          ), // Card
        ), // Padding
      ) // Center
    ); // Scaffold
  }
}
```

lista.dart:



```dart
import 'package:flutter/material.dart'; // Importa el paquete Material UI de Flutter / Imports Flutter Material UI package
import 'datos.dart'; // Importa el archivo de datos para acceder al modelo item / Imports data file to access item model
class Lista extends StatefulWidget{ // Define un StatefulWidget para mostrar la lista / Defines a StatefulWidget to display the list
  final List<item> items; // Variable final que recibe la lista de productos / Final variable receiving the products list
  const Lista({super.key,required this.items}); // Constructor con parámetro obligatorio de la lista / Constructor with required list parameter
  @override
  State<StatefulWidget> createState() {
    return _ListaState(); // Retorna la clase de estado / Returns the state class
  }
}

class _ListaState extends State <Lista>{ // Clase de estado para la lista de productos / State class for products list
  @override
  Widget build(BuildContext context) {
    final items = widget.items; // Obtiene la lista de elementos recibida / Gets the received items list
    return Scaffold( // Estructura visual principal de la pantalla / Main screen visual layout
      appBar: AppBar( // Barra superior de navegación / Top navigation app bar
        title: Text("Lista de Productos"), // Título de la AppBar / AppBar title
        backgroundColor: Colors.lightGreen, // Color de fondo verde claro / Light green background color
      ), // AppBar
      body: items.isEmpty // Valida si la lista está vacía / Validates if the list is empty
      ? Center( // Centra el contenido en pantalla / Centers content on screen
        child: Text(
          "Lista vacia", // Mensaje cuando la lista no tiene elementos / Message when list has no items
          style: TextStyle(
            color: Colors.red // Color rojo para el texto de advertencia / Red color for warning text
          ), // TextStyle
          ) // Text
        ) // Center
      : Center( // Centra la lista si contiene elementos / Centers list if it contains items
        child: Padding(
          padding: EdgeInsets.all(15), // Relleno general para la lista / General padding for the list
          child: ListView.builder( // Constructor eficiente para listas dinámicas / Efficient builder for dynamic lists
            itemCount: items.length, // Define la cantidad total de elementos / Defines total number of items
            itemBuilder: (context, index){
              final item = items[index]; // Obtiene el elemento actual de la lista / Gets current item from the list
              return Card( // Tarjeta contenedora para cada producto / Container card for each product
                margin: EdgeInsets.all(10), // Margen exterior de la tarjeta / Card external margin
                child:
                ListTile( // Estructura de elemento de lista / List tile layout structure
                  leading: Icon(Icons.production_quantity_limits), // Icono indicador de productos / Product indicator icon
                  title: Text(item.producto), // Muestra el nombre del producto / Displays product name
                  subtitle: Column( // Columna para mostrar la descripción y precio / Column to display description and price
                    children: [
                      Text(item.descripcion), // Muestra la descripción del producto / Displays product description
                      Text('Precio ${item.precio}') // Muestra el precio formateado / Displays formatted price
                    ],
                  ), // Column
                  trailing: IconButton( // Botón de acción a la derecha (eliminar) / Action button on the right (delete)
                    color: Colors.red, // Color rojo para el botón de eliminar / Red color for delete button
                    onPressed: (){
                      setState(() {
                        items.removeAt(index); // Elimina el elemento de la lista por su índice / Removes item from list by index
                      });
                    },
                    icon: Icon(Icons.delete) // Icono de papelera para eliminar / Trash can icon to delete
                  ), // IconButton
                ) // ListTile
                ); // Card
            }
          ) // ListView.builder
        ), // Padding
      ), // Center
    ); // Scaffold
  }
}
```


Datos
Escribe el producto
Escribe la descripcion
Escribe el precio
Agregar
Lista de Productos
Lista vacia
Lista de Productos
Refresco
Sin azucar
Precio 40